LAB-3758

Building Intelligent CI/CD Pipelines with IBM Bob

Hands-on lab guide

Javier Torres
jrtorres@us.ibm.com

Jordan Bond
jbond@ibm.com

Andy Madden
Andy.Madden@ibm.com

Cate Goldman
Cate.Goldman@ibm.com

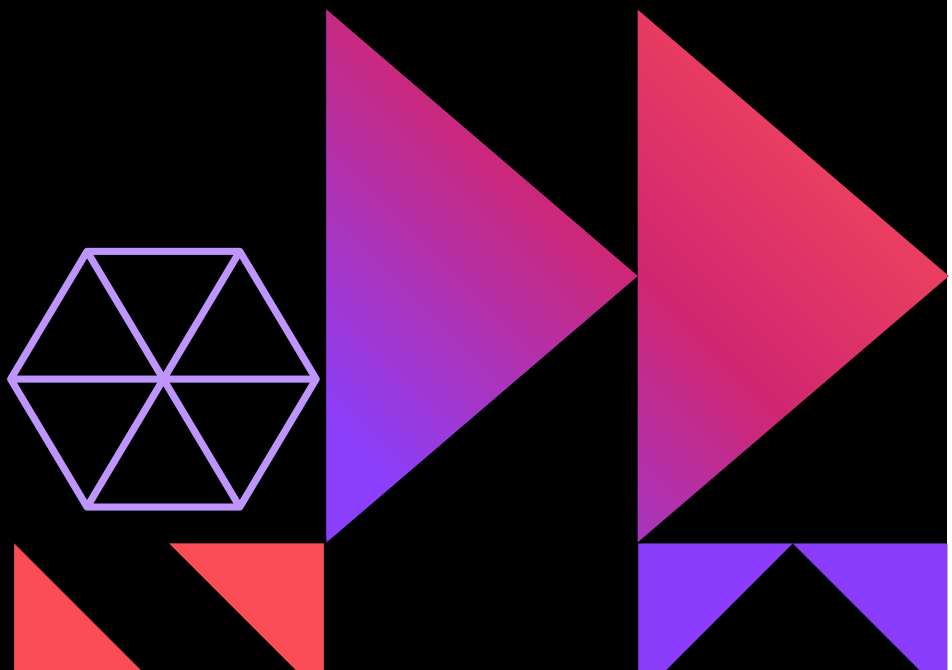


Table of Contents

Workshop Introduction 6

Lab Breakdown 6
Requirements, Resources and Tips 7

Part 0 - Setup 8

Task 0 — Ensure you can get to the lab environment 8
Task 1 — Fork the repository, set up Git access, and create your branch 8
1a. Fork the repository 8
1b. Create a GitHub PAT for your fork 9
1c. Clone your fork 10
1d. Open in Bob IDE 10
1e. Configure Git (if needed) 10
Task 2 — Create your Jenkins pipeline 11
Task 3 — Run it once to verify 11

Part 1 — PR / Git Diff Review with Bob 12

Before you start 12
Task 0 — Learn a little about the provided modes you'll be using 12
Task 1 — Add the *askBob* helper to your Jenkinsfile 13
Task 2 - Use Bob to analyze git diff 14
Task 3 - Create the *pipeline-git-diff-overview* mode 14
Task 4 — Build the *PR Review* stage 15
Task 5 — Push and watch 15

Part 2 — Unit Testing with Bob 17

Before you start 17
Task 1 — Create the *java-unit-test-mode* IDE mode 17
Task 2 — Use the mode to add a test 18
Task 3 — Create the *pipeline-test-failure-analyzer* mode 20
Task 4 — Build the *Unit Tests* stage 21
Task 5 — Push and watch 21

Part 3 — Security Vulnerability Analysis 23

What you'll build / test 23
Before you start 23
Task 1: Inject Security Vulnerabilities 23
Task 2: Security Analysis with Software Security Reviewer Mode 25
Task 3: SonarQube Security Scanning 27
Task 4: Add SonarQube & Grype Security Analysis Stage 28
Task 5: Push and Watch 31
Task 6: Restore Original Code 32
Information and Tips 33

Part 4 — Linting and Compliance with Bob 34

What you'll build 34
Before you start 34
Task 1 — Code Linting with Checkstyle 35
Task 2 — Deployment Artifact Linting 37
Task 3 — Create the Bob Linter mode and Pipeline Stage 37
Task 4 — Push and watch 39

Part 5 — Jenkins Pipeline Insights with MCP 41

What you'll build 41
Before you start 41
Task 1 — Jenkins MCP Server 42
Task 2 — Pipeline Information 45
Task 3 — Analysis and Insights 46

Wrap up 48

Accessing Lab Environment (Bob IDE) 49

Connect to your lab image 49
Launch Bob 50

Troubleshooting 52

All Pipeline Labs 52
Part 0 - Setup 52
Part 1 - PR / Git Diff Review with Bob 52
Part 2 - Unit Testing with Bob 53
Part 3 - Security Vulnerability Analysis 53
Part 4 - Linting and Compliance with Bob 54
Part 5 - Jenkins Pipeline Insights with MCP 54

Additional Resources 56

Feedback 57

Notices and disclaimers

© 2026 International Business Machines Corporation. No part of this document may be reproduced or transmitted in any form without written permission from IBM.

U.S. Government Users Restricted Rights — use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM.

This document is current as of the initial date of publication and may be changed by IBM at any time. Not all offerings are available in every country in which IBM operates.

Information in these presentations (including information relating to products that have not yet been announced by IBM) has been reviewed for accuracy as of the date of initial publication and could include unintentional technical or typographical errors. IBM shall have no responsibility to update this information.

This document is distributed "as is" without any warranty, either express or implied. In no event, shall IBM be liable for any damage arising from the use of this information, including but not limited to, loss of data, business interruption, loss of profit or loss of opportunity. IBM products and services are warranted per the terms and conditions of the agreements under which they are provided. The performance data and client examples cited are presented for illustrative purposes only. Actual performance results may vary depending on specific configurations and operating conditions.

IBM products are manufactured from new parts or new and used parts.
In some cases, a product may not be new and may have been previously installed. Regardless, our warranty terms apply."

Any statements regarding IBM's future direction, intent or product plans are subject to change or withdrawal without notice.

Performance data contained herein was generally obtained in a controlled,
isolated environments. Customer examples are presented as illustrations of how those customers have used IBM products and the results they may have achieved. Actual performance, cost, savings or other results in other operating environments may vary.

References in this document to IBM products, programs, or services does not imply that IBM intends to make such products, programs or services available in all countries in which IBM operates or does business.

Workshops, sessions and associated materials may have been prepared by independent session speakers, and do not necessarily reflect the views of IBM. All materials and discussions are provided for informational

purposes only, and are neither intended to, nor shall constitute legal or other guidance or advice to any individual participant or their specific situation.

It is the customer's responsibility to ensure its own compliance with legal requirements and to obtain advice of competent legal counsel as to the identification and interpretation of any relevant laws and regulatory requirements that may affect the customer's business and any actions the customer may need to take to comply with such laws. IBM does not provide legal advice or represent or warrant that its services or products will ensure that the customer follows any law.

Notices and disclaimers (Continued)

Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products. IBM does not warrant the quality of any third-party products, or the ability of any such third-party products to interoperate with IBM's products. **IBM expressly disclaims all warranties, expressed or implied, including but not limited to, the implied warranties of merchantability and fitness for a purpose.**

The provision of the information contained herein is not intended to, and does not, grant any right or license under any IBM patents, copyrights, trademarks or other intellectual property right.

IBM, the IBM logo, and ibm.com are trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at: www.ibm.com/legal/copytrade.shtml.

Workshop Introduction

This lab teaches you how to integrate IBM Bob, IBM's AI powered SDLC partner, directly into your CI/CD pipelines to bring AI into every step of software delivery, from development-time analysis in the IDE to automated stages in a Jenkins pipeline.

Every part of this lab follows the same loop: build a custom Bob mode in the Bob IDE, verify it locally, then ship it to the Jenkins pipeline as a new stage. By the end you will know how to create Bob custom modes, call Bob from a Jenkins pipeline as an AI agent, generate and triage unit tests, surface and explain security vulnerabilities, interpret findings from multiple linters, and monitor pipelines through Bob with MCP.

Lab Breakdown

Each part of this lab adds one stage to the same Jenkins pipeline and introduces a new Bob custom mode or integration pattern. The lab parts are stand-alone for the most part, so you can either do them sequentially or pick the portions you want to explore.

Part	Time	What you'll build	Use case
0 — Setup	5 min	Fork the repo, create a GitHub PAT and working branch, set up your Jenkins pipeline, and run a verification build	Lab environment ready
1 — PR / Git Diff Review	15 min	*askBob* helper function, *pipeline-git-diff-overview* mode, PR Review stage	Automated PR review and change analysis in every build
2 — Unit Testing	15 min	*java-unit-test-mode* for the IDE, *pipeline-test-failure-analyzer* mode, Unit Tests stage with JUnit publishing	Test generation and test failure diagnosis
3 — Security Analysis	20 min	Bob Findings on injected vulnerabilities, Software Security Reviewer mode, Security Analysis stage with SonarQube and Grype	Shift-left security with AI-assisted triage
4 — Linting and Compliance	15 min	Checkstyle, Hadolint, Checkov and KubeLinter, *pipeline-lint-analyzer* mode, lint stages that aggregate multi-tool output	Automated policy enforcement with prioritized findings
5 — Jenkins Insights with MCP	15 min	Jenkins MCP server in Bob to query builds, analyze failures and report on pipeline health.	Pipeline monitoring without leaving the IDE

Requirements, Resources and Tips

This lab will make use of the following GitHub repo - https://github.com/jrtorres/bob-cicd-lab3758, this is the source of truth for lab material and contains the application you will use throughout the lab. You will be forking this repo and using it as the SCM for the Jenkins pipeline & build. The GitHub repo contains the same instructions as this lab guide as well as more detailed explanations and additional lab challenges. Feel free to follow the labs in the repo or this document version.

General Requirements: In order to complete this lab, you will need the following:

- A GitHub account

Developer machine: You will be provided an individual environment with all the core requirements for the lab. This environment will be loaded with:

- Bob IDE, Git, Java 17, Maven, *uv*/uvx, a terminal and a modern web browser.

Shared resources: You will be given access to a shared set of resources.

- Jenkins – https://jenkins-jenkins.apps.itz-y6rg5f.infra01-lb.wdc07.techzone.ibm.com
    - You will be provided a username and password by the instructor.

Tips for the lab:

- Copy prompts exactly as shown in the lab guide and include @ file references when specified.
- Read Bob's response before running any generated code and ask follow-up questions when something is unclear.
- If you get stuck, see the Troubleshooting chapter at the end of this guide, or ask Bob.

Part 0 - Setup

Before you start the actual lab material, you will first set up **one Jenkins pipeline** that you'll reuse across the entire lab. Each part of the lab adds a new stage to *Jenkinsfile* on your branch; you push, click **Build Now**, and watch Bob do its thing.

Task 0 — Ensure you can get to the lab environment

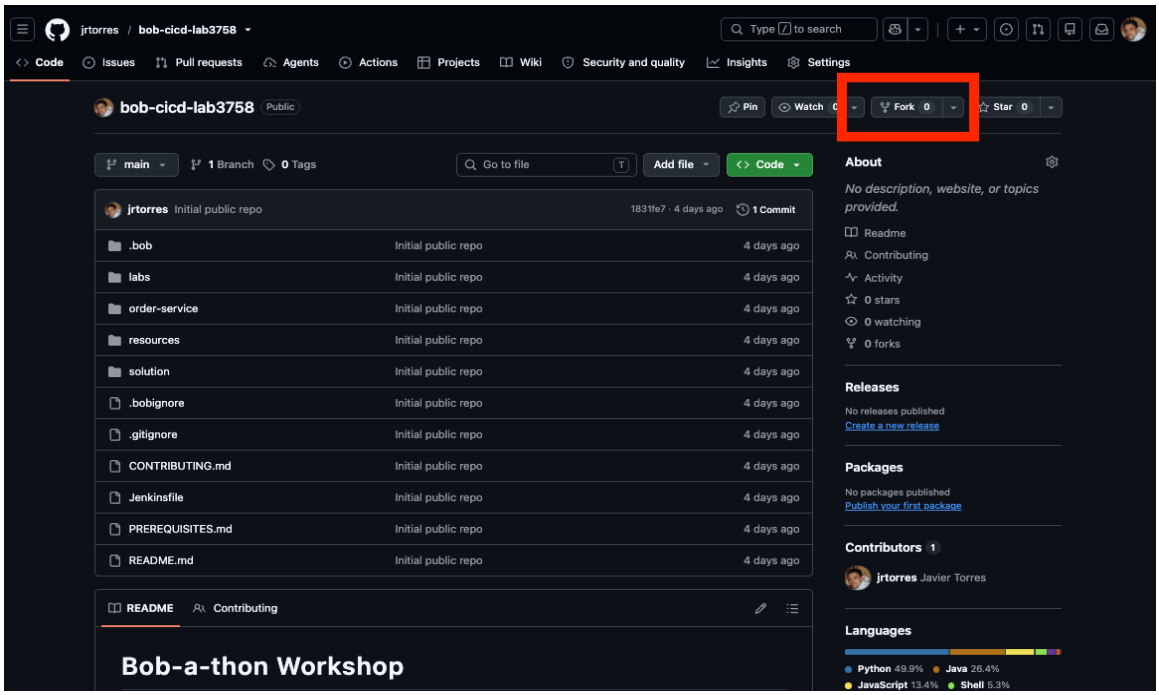
If you have not used a lab environment previously, follow the instructions in **Connect to your lab image** at the end of this document.

Task 1 — Fork the repository, set up Git access, and create your branch

1a. Fork the repository

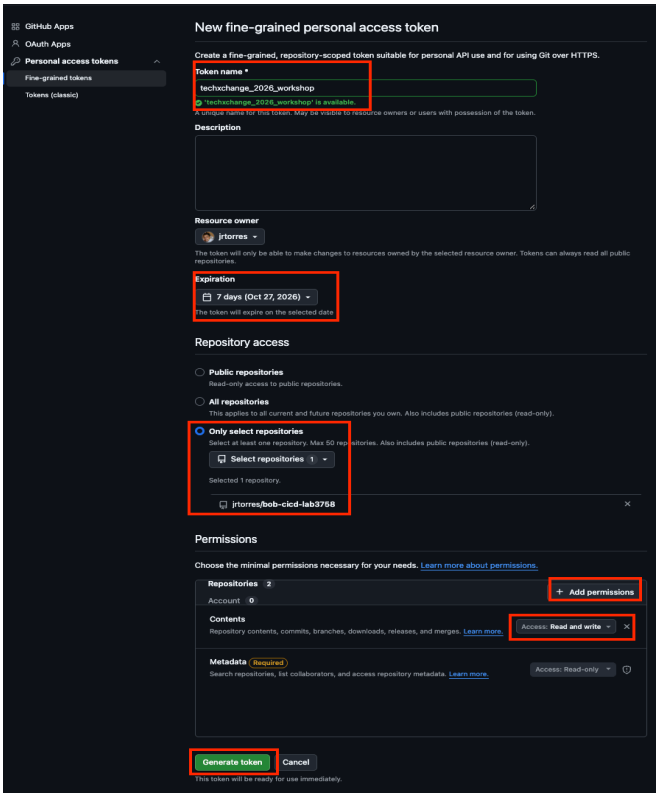
Your instructor will provide the repository URL for this workshop. Complete the following steps to fork your own copy of this repository:

1. Navigate to the repository URL provided by your instructor
2. Click the **Fork** button in the top-right corner
3. Select your account or organization as the destination
4. Wait for the fork to complete



1b. Create a GitHub PAT for your fork

You need a GitHub Personal Access Token (PAT) to authenticate when you *git push* your changes to your fork. To do this, you will create a **fine-grained** token limited to your fork, so it can't touch any of your other repositories.



1. Go to *https://github.com/settings/personal-access-tokens/new* (or **Settings → Developer settings → Personal access tokens → Fine-grained tokens → Generate new token**)
2. Set:
   - **Token name:** *techxchange_2026_workshop*
   - **Resource owner:** your own account
   - **Expiration:** 7 days
   - **Repository access: Only select repositories** → choose your fork (*bob-cicd-lab3758*)
3. Under **Permissions → Repository permissions**, Click the *Add permissions* button and select **Contents**.
4. Ensure **Contents** is set to **Read and write**. Leave everything else as *No access* (**Metadata: Read-only** is added automatically).
5. Click **Generate token**.
6. **Copy the token now** — GitHub won't show it to you again. Save it somewhere secure (a password manager is recommended).

> **Fine-grained tokens unavailable?** If your account or organization doesn't allow them, create a classic token instead: **Generate new token (classic)** at *https://github.com/settings/tokens*, check only ***repo***, and

> set a 7-day expiration. Classic tokens cover all of your repositories, so revoke it as soon as the workshop ends.

1c. Clone your fork

You will clone your fork to the Bob IDE desktop machine.

1. Open a terminal on the desktop machine and run the following commands:

```
# Replace <YOUR_USERNAME> with your actual GitHub username
git clone https://github.com/<YOUR_USERNAME>/bob-cicd-lab3758.git
cd bob-cicd-lab3758
```

2. Verify the remote branch is pointing to your own fork

```
git remote -v
# Should show your fork as 'origin'
```

1d. Open in Bob IDE

1. Launch Bob IDE
2. Open the cloned repository by using the menu options: **File → Open Folder**, select the *bob-cicd-lab3758* directory, and click **Open**
3. Verify Bob can see the repository files in the Explorer sidebar

1e. Configure Git (if needed)

1. If this is your first time using Git on this machine, configure your identity.
2. You can run the following commands from an existing terminal, or open a terminal directly within the IBM Bob IDE:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

3. Each *git push* in the labs asks for your GitHub username and your PAT (as the password). To avoid re-entering the PAT on every push, run this once **before** your first push:

```
git config --global credential.helper 'cache --timeout=10800'
```

4. Git will now remember your credentials for 3 hours. You enter the PAT on your first push, and later pushes won't ask again.

> **Note:** The cache is held in memory only. Nothing is written to disk. It's lost on reboot or after 3 hours, and you'll just be prompted for the PAT again.
>
> **Prefer not to cache?** Skip the command above. Git will then ask for your username and PAT on every *git push* (about 7–8 times across the labs), and you paste the PAT as the password each time.

5. Next, you will create your working branch, where all your updates throughout the labs will be commited. Run the following commands:

```
# Create your branch and push it
git checkout -b user1-labs
git push -u origin user1-labs
```

> ***Note:***
>
> – *Replace* user1-labs *with your actual username (e.g.,* user7-labs*). You'll push to this branch after every lab exercise.*
>
> – *Git prompts for your username and password. Enter your GitHub username and paste your PAT as the password.*

Task 2 — Create your Jenkins pipeline

To setup Jenkins, you will create a pipeline inside a specific *user#* folder, not at the Jenkins root. If you create it at the root, you won't have permission to save it.

1. Open the Jenkins URL you were provided in a web browser
2. Click your *user#* folder.
3. Inside the folder, click **New Item** in the left sidebar.
4. Enter a name — something like *user1-pipeline*.
5. Select **Pipeline** from the type list, then click **OK**.
6. You land on the config page. Scroll to the **Pipeline** section at the bottom and fill in:

Field	Value
Definition	*Pipeline script from SCM*
SCM	*Git*
Repository URL	paste the GitHub repo URL from your fork
Credentials	leave as *- none -* (your fork is public)
Branch Specifier	*/user1-labs* — replace with **your** branch name
Script Path	*Jenkinsfile*

7. Click **Save**. You land on your new pipeline's page.

Task 3 — Run it once to verify

1. Click **Build Now** in the left sidebar.
2. A build appears under "Build History" — click its number, then **Console Output**.
3. It is **expected** that the *Checkout* stage completes and the console prints your workspace contents (*.bob/*, *Jenkinsfile*, *order-service/*, etc.). The pipeline finishes *SUCCESS* with no other stages yet — that's correct; you'll add stages lab by lab.

Part 1 — PR / Git Diff Review with Bob

In this portion of the lab you will complete the following:

- Add a ***askBob*** **helper function** — your pipeline's reusable interface to Bob. One helper, optional second argument for pinning a custom mode. Every subsequent lab calls this same helper.
- **A built-in look at the diff** — before writing any pipeline code, you'll use the Bob IDE to analyze the git diffs. The output is fine but unstructured — that gap is what motivates the next part.
- Create a **custom Bob mode** (*pipeline-git-diff-overview*) built with **Mode Writer**, this mode pins the output shape and the read-only permissions a CI mode should have. Your pipeline stage will hand Bob a one-line prompt and the mode does the rest.
- Add a ***PR Review*** **stage**, which uses the **Jenkins Pipeline Integration** mode (provided in this repo) to write a stage that calls *askBob* with your new mode and archives the result.

By the end, every push triggers a structured PR review in your Jenkins console — and you've practiced the prompt-then-mode loop you'll repeat for unit tests, security scans, and linting in the labs that follow.

Before you start

- ☐ Part 0 complete (branch created, pipeline pointed at it, one successful *Checkout*-only build)
- ☐ You're on your working branch (e.g. *user1-labs*)

Task 0 — Learn a little about the provided modes you'll be using

Every lab leans on two custom modes that ship with this repo:

- **Mode Writer** (*mode-writer*) — used in most labs to create new custom modes. By default it may only write the mode definition into *.bob/custom_modes.yaml*, but you can ask it explicitly to generate **rules files** (or add them yourself later) to sharpen the mode's behavior beyond what fits in a single YAML block.
- **Jenkins Pipeline Integration** (*jenkins-bob-integration*) — used in every lab that touches the Jenkinsfile. It's been pre-loaded with the specifics of this workshop's pipeline environment (containers, the *askBob* helper, our credential pattern) so you spend time on the lab's goal instead of debugging environment quirks.

Take a quick glance at the rules files behind each mode to see how customizable a Bob mode can get:

- *.bob/rules-mode-writer/* (4 files)
- *.bob/rules-jenkins-bob-integration/* (8 files)

Task 1 — Add the *askBob* helper to your Jenkinsfile

Every stage of the pipeline that invokes Bob has to do the same two things: step into *container('bob')* and run *bob ...*. Factor that out **once** and each stage just passes a prompt and gets back the analysis string to do whatever it wants with.

1. The code for this function is shared below. Paste the helper into your *@Jenkinsfile* at the bottom, **outside** the *pipeline { }* block:

```
// ── Helper: ask Bob, optionally with a specific custom mode ──────────────────
// Writes the prompt to a tempfile in the shared workspace and runs `bob` in
// the bob container.
// Returns the analysis as a string. Using a tempfile (instead of inlining the
// prompt on the command line) avoids shell-escaping issues when the prompt
// contains quotes, backticks, or newlines — common with diffs.
def askBob(String prompt, String mode = null) {
  container('bob') {
    def promptFile = ".bob-prompt-${System.currentTimeMillis()}.txt"
    writeFile file: promptFile, text: prompt

    def modeFlag = mode ? "--mode ${mode}" : ""
    def analysis = sh(
      script: """cat ${promptFile} | bob run ${modeFlag} --accept-license""",
      returnStdout: true
    ).trim()

    sh "rm -f ${promptFile}"
    return analysis
  }
}
```

2. Once you've pasted the helper into your *@Jenkinsfile*, save the file, start a new task and switch to *Ask* mode. Ask bob:

```
Can you explain to me the key parts of the askBob function in @Jenkinsfile
```

3. In the pipeline, we will invoke Bob using the command: *bob run --mode ${modeFlag}*. What this command does is the following:
   - ***bob*** — the Bob CLI, available in the *bob* container.
   - ***${modeFlag}*** — placeholder for the optional *--mode <slug>* flag. When *askBob* is called with a second argument (a custom mode slug, e.g. *pipeline-git-diff-overview*), this expands to *--mode pipeline-git-diff-overview* so Bob applies that mode's rules and tools. When no mode is passed, it's empty and Bob uses its built-in default. Custom modes are how you give a stage stable, structured output without spelling out the format in every prompt — you'll build one in Part 5.
4. With that understanding of how this helper works, commit your changes to the pipeline. Run the following commands in the terminal :

```
git add Jenkinsfile
git commit -m "Add askBob helper to @Jenkinsfile"
git push
```

Task 2 - Use Bob to analyze git diff.

1. Start a new task and switch to *Agent* mode (bottom left of the Bob chat panel). Ask Bob to analyze the diff. Prompt Bob with:

```
Bob, analyze the diff between my branch and main.
```

2. You may need to approve the tool calls Bob tries to make.
3. Bob can analyze the diff using the *obtain_git_diff* tool, the Github MCP server or even with Github CLI. This is something you can specify in your *AGENTS.md* file once you gain a preference.
4. This overview of the diff is fine, but we can implement custom modes to get more detailed and better formatted information.

Task 3 - Create the *pipeline-git-diff-overview* mode

The overview Bob gave in Task 2 was unstructured — fine for one-off testing. A custom mode pins the output shape and the read-only permissions a CI mode should have.

1. Start a new task and switch to **Mode Writer** mode. Mode Writer turns a prompt into a reusable custom mode — you describe the behavior you want once, and Bob applies it on every future invocation of that mode.

2. The prompt below is a starter for a senior-developer-style diff overview — a quick risk-oriented glance, not a full code review. Treat it as a base. Anything you want every PR overview to do (output format, patterns to flag, noise to skip, the tone you want Bob to take) belongs in this prompt.

```
Write a custom mode with slug `pipeline-git-diff-overview`. Append it to @.bob/custom_modes.yaml — don't overwrite anything else.

Job: senior dev glancing at a PR's git diff. Quick risk-oriented overview, not a full review. For each notable change:
 - 1–2 sentences on what changed
 - Risk: high / medium / low
 - Watch for: null-safety, concurrency, behavior changes, security surface, perf hot paths, missing tests

Output: plain text for a Jenkins console (no ANSI, no markdown tables). Sections: Summary, Risk, Watch for. Short — if nothing notable changed, say so in one sentence.

Tool groups: read only.
```

> *Note: Read-only is deliberate — a pipeline mode should do the minimum it needs to. No IDE restart needed: Bob loads* custom_modes.yaml *fresh from the workspace on every pipeline run.*

3. Run the prompt and answer any questions Bob might have about the implementation.

Task 4 — Build the *PR Review* stage

Now that you have created a custom mode, let's add the new stage to the *Jenkinsfile*.

1. Start a new task and switch to the provided **Jenkins Pipeline Integration** mode. This mode was provided to minimize configuration issues with the environment. It knows about *askBob*, the build-tools container, output archiving, and *catchError* resilience patterns. Its rules live in *.bob/rules-jenkins-bob-integration/* if you want to look.
2. Paste the following prompt:

> *Add a "PR Review" stage to @Jenkinsfile right after the Checkout stage. The stage should:*
>
> *- Compute the PR diff and write it to git-diff.txt*
> *- Call askBob with the pipeline-git-diff-overview mode and a short prompt asking it to read git-diff.txt and produce the overview*
> *- Save the analysis to bob-pr-review.md and archive it as a build artifact*

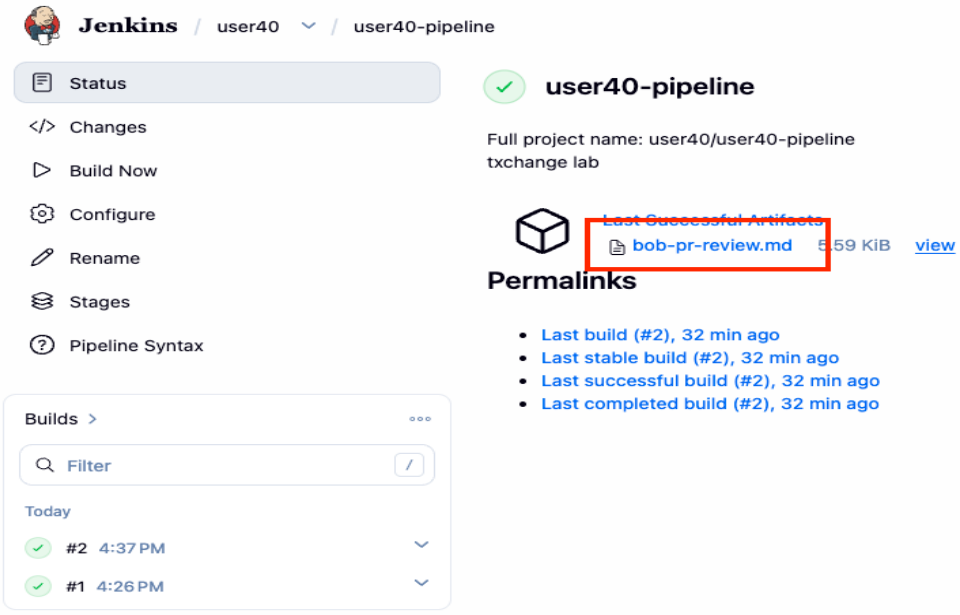
3. Provide feedback to Bob where needed and consider turning on auto-approval for some tool calls.

Task 5 — Push and watch

1. Next, lets commit our changes, from the terminal, run:

```
git add Jenkinsfile .bob/
git commit -m "Lab 1 — PR review stage with askBob helper + pipeline-git-diff-overview mode"
git push
```

2. In Jenkins, click **Build Now** on your pipeline and watch the console. We expect the following:
   - *Checkout* stage turns green
   - *PR Review* stage runs next. The console shows your stage's banner (*Bob — PR Review* or however you wrote it) with Bob's summary / risk / watch-for output between the banner lines
   - Build page lists *bob-pr-review.md* under **Build Artifacts**
   - Pipeline ends SUCCESS
3. Open the archived artifact from the build page and you've got a persistent record of Bob's take on that commit — searchable in Jenkins's build history.


Jenkins / user40 / user40-pipeline
Status
Changes
Build Now
Configure
Rename
Stages
Pipeline Syntax
Builds
Filter
Today
#2 4:37 PM
#1 4:26 PM
user40-pipeline
Full project name: user40/user40-pipeline
txchange lab
Last Successful Artifacts
bob-pr-review.md 5.59 KiB view
Permalinks
Last build (#2), 32 min ago
Last stable build (#2), 32 min ago
Last successful build (#2), 32 min ago
Last completed build (#2), 32 min ago

Part 2 — Unit Testing with Bob

In this portion of the lab, you'll add automated unit-test analysis to the pipeline using the same prompt-then-mode loop you practiced in Part 1 — one custom mode for writing tests in your IDE, another for diagnosing failures in CI.

You will reuse the following from Part 1:

- **The *askBob* helper** — already in your Jenkinsfile. Lab 2's stage calls it with the new test-failure-analyzer mode.
- **The *Jenkins Pipeline Integration* mode** — same mode you used to write the PR Review stage. It already knows how to wire up askBob, archive output, and use *catchError* for resilience.

You will build the following:

- **A custom Bob mode for writing unit tests** (*java-unit-test-mode*) — built with **Mode Writer**; an IDE specialist that knows JUnit 5 + Mockito + the conventions of this repo's existing tests.
- **An IDE-side test addition** — using your new mode, find an uncovered edge case and write a test for it. Bob runs the test for you and iterates if it fails.
- **A custom Bob mode for analyzing test failures** (*pipeline-test-failure-analyzer*) — read-only pipeline mode that diagnoses failing tests and proposes fixes.
- **The *Unit Tests* stage** — uses the **Jenkins Pipeline Integration** mode to write a stage that runs *mvn test*, publishes JUnit results, and (only when something fails) hands the surefire reports to Bob for analysis.

By the end, a failing test won't just turn the build yellow — Bob will explain what broke, why, and how to fix it, all in plain English in your Jenkins console.

Before you start

- ☐ Lab 1 complete (askBob helper added to your Jenkinsfile)
- ☐ You're on your working branch (e.g. *user1-labs*)
- **Reset your Jenkinsfile first.** Remove the stages you added in previous labs, keeping only the *Checkout* stage and the *askBob* helper at the bottom. Stacking every lab's stages stretches a single build past 15 minutes — clearing them keeps iteration fast and the logs readable. Use *solution/Jenkinsfile.start* as the reference shape.

Task 1 — Create the *java-unit-test-mode* IDE mode

1. Start a new task and switch to **Mode Writer** mode.
2. The prompt below is a starter for a Java unit-test specialist. Treat it as a base, not a fixed script: anything you want every test you write to follow (naming convention, *@DisplayName*, given/when/then comments, fixture builders, parameterized tests) belongs in this prompt rather than the per-test call.

> *Write a custom mode with slug `java-unit-test-mode`. Append it to @.bob/custom_modes.yaml — don't overwrite anything else.*
>
> *Job: write Java unit tests for this Spring Boot application. JUnit 5 + Mockito + AssertJ.*
> *Before writing a new test, read the related existing tests under @order-service/src/test/ to match this repo's conventions (assertion library, mocking style, naming pattern, fixture setup).*
>
> *Tool groups:*
> *- read*
> *- edit*
>
> *Add some instructions files for the mode. The instruction files should be Markdown files, not xml.*

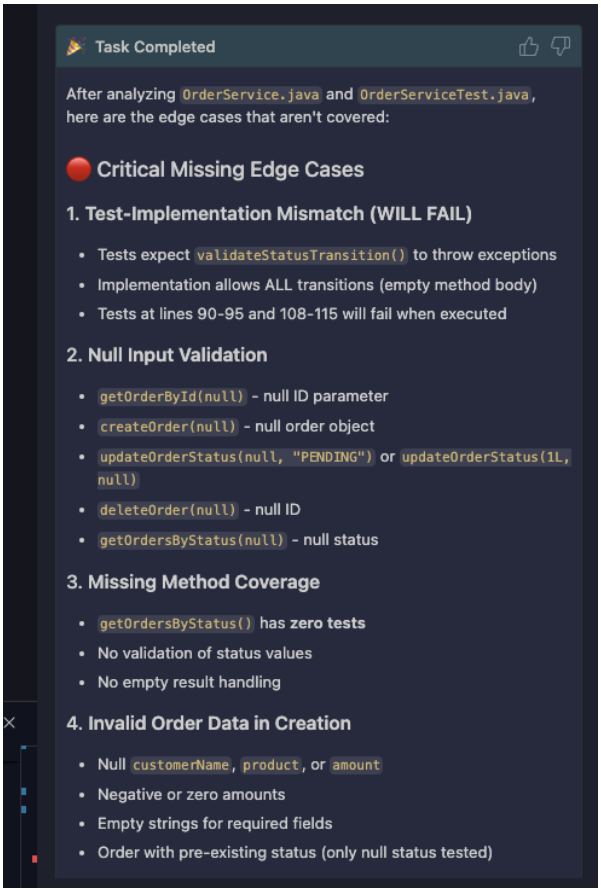
3. Notice the prompt above tells the **Mode Writer** mode to write Markdown files instead of XML this time. Both are acceptable practice with Bob.
4. Feel free to explore the Markdown files and see what Bob decided to generate.

Task 2 — Use the mode to add a test

1. In a new task, switch to **Ask** mode and ask:

> *Read @OrderService.java and @OrderServiceTest.java. What edge cases aren't covered?*

2. Bob will show you something similar to this:



3. Pick one of Bob's suggestions, like *getOrderById(null) - null ID parameter*.
4. Then switch to **Java Unit Test** mode (your new mode) in the same task, and ask:

> *Write a test for [the edge case] and add it to @OrderServiceTest.java."*

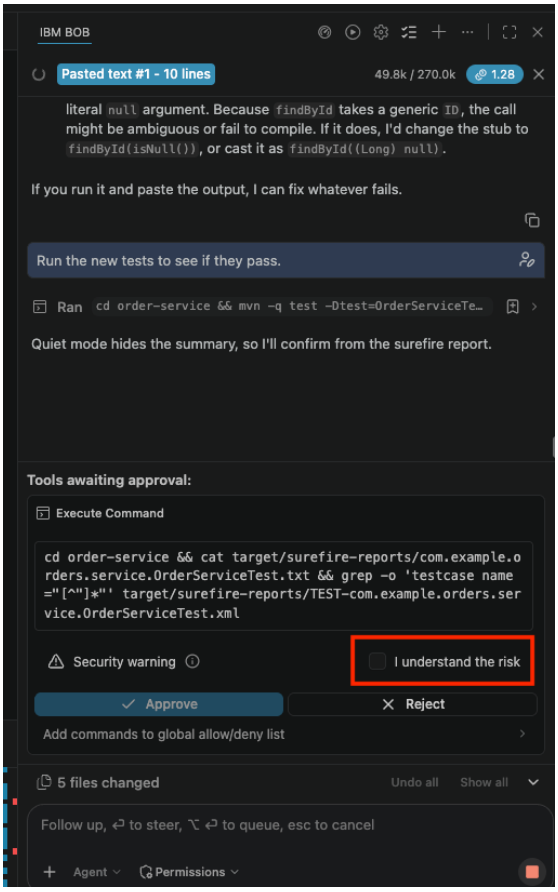
5. After Bob writes the tests, ask Bob to run the test (you may need to install maven).

> *Run the new tests to see if they pass."*

6. Bob will ask to switch modes because we didn't give the unit test mode the *command* tool group.

   Note: If Bob does not ask to switch modes, simply switch to *Agent* mode and run the request again.

   Bob will run the tests using Maven. If prompted with a security warning, click the *I understand the risk* box and approve.



Task 3 — Create the *pipeline-test-failure-analyzer* mode

Your test-writer mode is great for the IDE — it edits files and writes new tests. The pipeline needs something different: a read-only mode that diagnoses test failures and proposes fixes, with output formatted for a Jenkins console.

1. Start a new task and switch to **Mode Writer** mode. The prompt below is a starter — anything you want every test-failure diagnosis to do (output format, what to surface, what to skip as noise, tone) belongs here, not in the per-stage call.

> *Write a custom mode with slug `pipeline-test-failure-analyzer`. Append it to @.bob/custom_modes.yaml — don't overwrite anything.*
>
> *Job: diagnose JUnit / Surefire test failures and propose fixes. Read the failure reports plus the relevant source files under @order-service/src/. For each failed test:*
> *- One sentence on what failed and where*
> *- Likely root cause (assertion mismatch, NPE, mock misconfiguration, timeout, wrong exception type, etc.)*
> *- Suggested fix with a short code snippet*
> *- Group related failures if they share a root cause*
>
> *Output: plain text for a Jenkins console (no ANSI, no markdown tables). Sections: Summary, Failures, Suggested fixes. Short — if a single line says it, say it in a single line.*

> *Tool groups: read only.*

> Note: Read-only is deliberate — a pipeline mode should do the minimum it needs to. No IDE restart needed: Bob loads *custom_modes.yaml* fresh from the workspace on every pipeline run.

2. We didn't explicitly tell Bob to create rules files here, so it may not have. If you feel you want more detailed instructions for the mode, ask Bob in Mode Writer mode to expand the mode into rules files.

Task 4 — Build the *Unit Tests* stage

1. Start a new task and switch to the **Jenkins Pipeline Integration** mode. This is the same mode you used in Lab 1— it knows about *askBob*, the build-tools container, output archiving, and *catchError* resilience patterns. Rules live in *.bob/rules-jenkins-bob-integration/* if you want to look.
2. Paste the following prompt:

> *Add a "Unit Tests" stage to @Jenkinsfile right after the PR Review stage. The stage should:*
>
> *- Run `mvn test` inside order-service/ in the build-tools container, capturing the result so the pipeline continues even if tests fail*
> *- Publish JUnit results from order-service/target/surefire-reports/*.xml*
> *- If tests failed, call askBob with the pipeline-test-failure-analyzer mode and a short prompt asking Bob to analyze the surefire reports under order-service/target/surefire-reports/ alongside the relevant source files under order-service/src/*
> *- Save the analysis to bob-test-analysis.md and archive it as a build artifact*

Task 5 — Push and watch

The Unit Tests stage only triggers Bob's analysis when a test fails — so to see it in action, deliberately break a test before pushing.

1. On line 48 of *order-service/src/test/java/com/example/service/OrderServiceTest.java*, we have the following assertion:

> *assertThat(orders).hasSize(1);*

2. Change that to:

> *assertThat(orders).hasSize(2);*

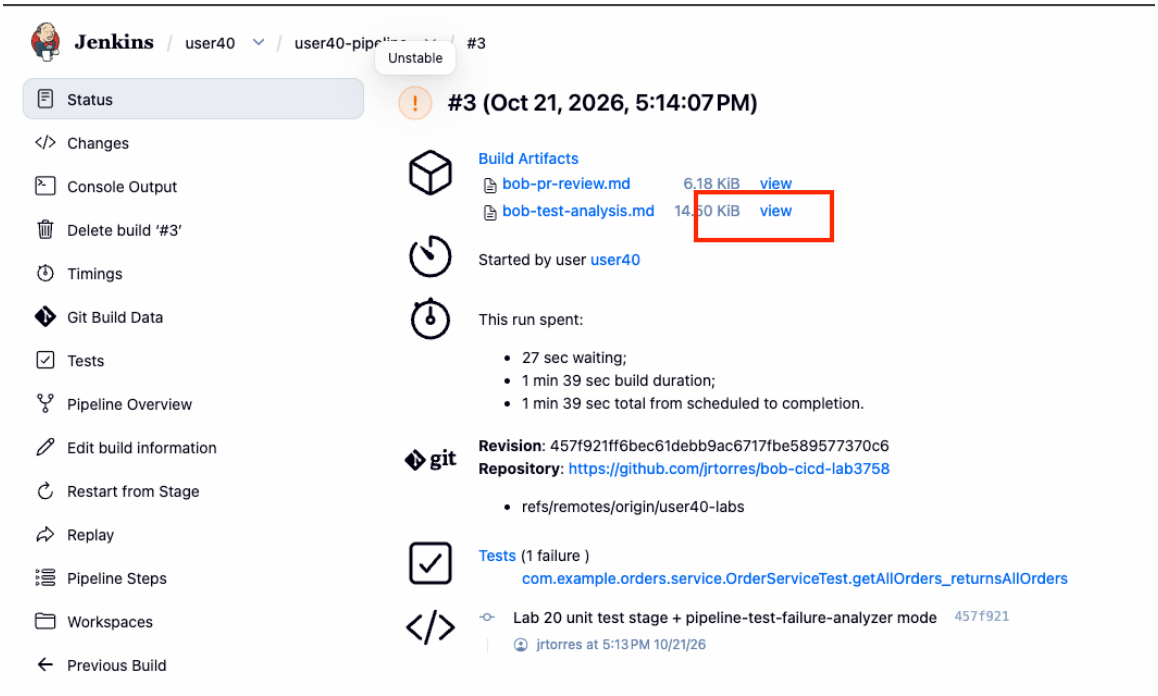
3. Commit and push:

> *git add Jenkinsfile .bob/ order-service/*
> *git commit -m "Lab 2 — Unit Tests stage + pipeline-test-failure-analyzer mode"*
> *git push*

4. In Jenkins, click **Build Now** and watch the console. You should see the following:
   - *Checkout* and *PR Review* turn green
   - *Unit Tests* turns **yellow (UNSTABLE)** — the failing test is recorded, but the pipeline continues
   - Jenkins's Test Results link surfaces the failure with the exact assertion message
   - Bob's analysis stage runs; the console shows root cause + suggested fix between banner lines
   - *bob-test-analysis.md* appears under **Build Artifacts**

- Pipeline ends UNSTABLE (yellow), not red

5. Open the archived artifact for a persistent record of Bob's diagnosis. Look into the build artifact and see if Bob found the failed test you added.



6. Once you've seen it work, revert the broken test and push again to get back to green.

You're ready to proceed to the next portion of the lab, security analysis.

Part 3 — Security Vulnerability Analysis

In this portion of the lab, you will learn how to perform security analysis with Bob Findings, run automated scans with SonarQube, and integrate comprehensive security checks into a CI/CD pipeline.

You'll inject security vulnerabilities into the order-service, analyze them with Bob's real-time findings and a comprehensive security review mode, run automated SonarQube scans, integrate security checks into your Jenkins pipeline, and learn proper Git workflows for managing experimental changes.

What you'll build / test

- **Security vulnerability injection** — use a script to inject 6 types of vulnerabilities into OrderService.java and observe Bob's real-time findings as they appear in your IDE.
- **Comprehensive security analysis** — import and use the Software Security Reviewer mode to generate a full application security assessment covering code, infrastructure, compliance, and remediation roadmap.
- **SonarQube integration** — generate a SonarQube token, run automated security scans, and review detailed vulnerability reports with metrics and issue tracking.
- **Enhanced Jenkins pipeline** — add a comprehensive security stage to your Jenkinsfile that runs SonarQube scans and generates security reports on every build.
- **Git workflow best practices** — use *git restore* to discard experimental vulnerability injections and maintain a clean working directory before pushing changes.

By the end, every push triggers automated security scanning in your Jenkins pipeline, and you've practiced using Bob's security analysis capabilities across multiple tools and workflows, including proper Git hygiene for managing temporary code changes.

Before you start

- ☐ Lab 1 complete (askBob helper added to your Jenkinsfile)
- ☐ You're on your working branch (e.g. *user1-labs*)
- ☐ Your instructor will provide you the credentials for SonarQube
- **Reset your Jenkinsfile first.** Remove any stages you added in previous labs, keeping only the *Checkout* stage and the *askBob* helper at the bottom. Stacking every lab's stages stretches a single build past 15 minutes — clearing them keeps iteration fast and the logs readable. Use *solution/Jenkinsfile.start* as the reference shape.

Task 1: Inject Security Vulnerabilities

1. Let’s first review the *Bob Findings* before injecting some vulnerabilities

2. Select **Bob Findings** at the bottom of the screen. You will observe that it is currently empty. This is expected, as no vulnerabilities have been injected into the file yet, so Bob has no findings to display.
3. For the purposes of this lab, we will inject some vulnerabilities into our sample application.
4. Ensure you are in **Agent Mode** before prompting Bob with the following:

```
Bob run script inject_vulnerabilities.sh
```

5. Running the script:
   - Modifies *order-service/src/main/java/com/example/orders/service/OrderService.java*
   - Injects vulnerable dependencies into *order-service/pom.xml*
   - Injects 9 types of vulnerabilities:
   - **Code Vulnerabilities (6):**
     - Hardcoded credentials (BACKUP_DB_PASSWORD, LEGACY_API_KEY)
     - Insecure logging (System.out.println)
     - Weak cryptography (MD5)
     - Stack trace exposure (printStackTrace)
     - Weak random (java.util.Random)
     - Information exposure (detailed error messages)
   - **Dependency Vulnerabilities (3):**
     - Log4j 2.14.1 - CVE-2021-44228 (Log4Shell - Critical)
     - Jackson Databind 2.12.3 - CVE-2020-36518 (High severity)
     - SnakeYAML 1.30 - CVE-2022-1471 (Critical severity)
6. The three vulnerable dependencies introduce **multiple high and critical severity CVEs** that will be detected by Grype during the security scan section of the lab. This will be used to demonstrate how dependency vulnerabilities are identified, assessed, and prioritized in the pipeline.
7. The **Expected Output** of the script should be similar to the following:

```
🔧 Vulnerability Injection Script
==================================

🔧 Injecting vulnerabilities into OrderService.java...
  ✅ Java vulnerabilities injected

🔧 Injecting vulnerable Log4j dependency into pom.xml...
  ✅ Vulnerable Log4j 2.14.1 dependency injected (CVE-2021-44228)

✅ Vulnerability injection complete!

📍 Modified files:
  - order-service/src/main/java/com/example/orders/service/OrderService.java
  - order-service/pom.xml

🔍 Injected vulnerabilities:
  1. Hardcoded credentials (BACKUP_DB_PASSWORD, LEGACY_API_KEY)
  2. Insecure logging (System.out.println with sensitive data)
  3. Weak cryptography (MD5 in generateOrderVerificationCode)
  4. Stack trace exposure (printStackTrace in multiple methods)
  5. Weak random (java.util.Random in generateTrackingNumber)
  6. Information exposure (detailed error messages)
  7. Vulnerable dependency (Log4j 2.14.1 - CVE-2021-44228)
```

> *8. Vulnerable dependency (Jackson Databind 2.12.3 - CVE-2020-36518)*
> *9. Vulnerable dependency (SnakeYAML 1.30 - CVE-2022-1471)*

8. Now that vulnerabilities have been injected, let's see what Bob detects in real-time. **Open the modified file:**
   - Navigate to: *order-service/src/main/java/com/example/orders/service/OrderService.java*
9. Click **"Bob Findings"** at the bottom of the screen.
   - Bob will analyze the file and identify security issues in real-time
   - **You should see 4 critical findings** displayed in the Bob Findings panel

> ***Note:*** *Bob Findings should update with its security findings, if it does not, try to close and reopen the* OrderService.java *file or restart Bob*

10. Bob Findings intelligently prioritizes and displays the **most critical security issues** that require immediate attention. We expect to find 4 Critical Findings in OrderService.java
    - 🔴 **Scan (Secrets): Secret Keyword:19** - Hardcoded credential detected
    - 🔴 **Scan (Secrets): Secret Keyword:20** - Hardcoded credential detected
    - 🔴 **Scan (Vulnerabilities): Integer.toHexString...** - Weak cryptography implementation
    - 🔴 **Scan (Vulnerabilities): Detected MD5 hash...** - Insecure hash algorithm usage

    > ***Note: Why 4 findings instead of 9 ?*** While the script injected 9 types of vulnerabilities, Bob Findings focuses on **critical security issues** that pose the highest risk:

Vulnerability Type	Detected by Bob Findings?	Reason
Hardcoded credentials	✅ Yes (2 findings)	**CRITICAL** - Direct security breach risk
Weak cryptography (MD5)	✅ Yes (2 findings)	**CRITICAL** - Cryptographically broken algorithm
Insecure logging (System.out)	⚠️ Lower priority	Code smell, not critical vulnerability
Stack trace exposure	⚠️ Lower priority	Context-dependent risk
Weak random (java.util.Random)	⚠️ Lower priority	May be flagged differently
Information disclosure	⚠️ Lower priority	Context-dependent risk
Vulnerable dependencies (3)	⚠️ Not in code scan	Detected by dependency scanners (SonarQube/Grype)

    > **This is expected behavior!** Bob Findings prioritizes critical security issues (secrets and cryptographic vulnerabilities) over code quality issues. The comprehensive Security Analysis Report in Part 2 will capture all issues, while Bob Findings focuses on what developers need to fix immediately.

11. This gives you a preview of what Bob detects at the code level before we perform a comprehensive security analysis in Part 2.

Task 2: Security Analysis with Software Security Reviewer Mode

Before importing the *Software Security Reviewer* mode, take a moment to understand what it does and how it differs from Bob Findings.

1. Ask Bob to explain. In *Ask* mode, prompt Bob with the following:

> *Read @software-security-reviewer.yaml and explain what the Software Security Reviewer mode does and how it differs from Bob Findings. What types of analysis does it perform?*

2. Bob should return an explanation that covers the following:
   - **Software Security Reviewer** is a specialized mode for comprehensive application security audits
   - It goes beyond code-level analysis to include infrastructure, configuration, and compliance
   - Performs multi-layer security assessment across the entire application stack
   - Generates detailed reports with remediation roadmaps and compliance mapping
3. Let's import the specialized **Software Security Reviewer** mode. Click the **Settings icon** (⚙) in the top right corner of Bob
4. Click **"Modes"** in the settings menu, then Click the **"Import"** button
5. Select the file: *labs/lab3/software-security-reviewer.yaml* and Click **"Import"** to confirm.
6. Verify the mode was imported. You should see **"🛡️🔐 Software Security Reviewer"** in your modes list
7. Close the settings panel
8. Select the **"🛡️🔐 Software Security Reviewer"** mode from the dropdown
9. Now let's have Bob perform a full security audit of the order-service application. Prompt Bob with the following:

> *Evaluate order-service for vulnerabilities, insecure patterns, misconfigurations, and compliance gaps and generate an analysis report.*

10. The approach Bob takes may vary. Sometimes Bob creates a TODO list to track the analysis process, other times it proceeds directly to scanning. Both approaches are valid. In either case, Bob will:
    - Scans all application files (Java code, configuration, Dockerfile, K8s manifests)
    - Identifies security vulnerabilities and insecure patterns
    - Analyzes compliance gaps (PCI DSS, OWASP Top 10)
    - Evaluates misconfigurations in infrastructure
    - Generates comprehensive security report: *SECURITY_ANALYSIS_REPORT.md*
11. Review the *SECURITY_ANALYSIS_REPORT.md* report to understand each finding, its impact, and remediation steps. It should include:
    - Detailed findings with CVSS scores, CWE mappings, and code examples
    - Specific remediation steps for each vulnerability
    - OWASP Top 10 (2021) compliance matrix
    - PCI-DSS compliance status
    - Prioritized remediation roadmap (4 phases over 6 weeks)
    - Supply chain & infrastructure security recommendations
    - CI/CD pipeline security gates
12. Between the Bob Findings and this Software Security Reviewer mode we now have a comprehensive security assessment that together provides:

- 🔍 Real-time detection (Bob Findings: 4 critical items) + Comprehensive analysis (Security Reviewer: 17 total findings)
- 💻 Critical code issues + Full code & configuration security
- ⚡ Developer feedback + Security team reporting
- 🎯 Immediate fixes + Strategic remediation planning

Task 3: SonarQube Security Scanning

1. Click **"Start New Task"** in Bob to begin a fresh conversation. This ensures Bob focuses on the SonarQube scanning workflow without context from the previous security analysis.
2. Switch to **Agent Mode** before proceeding with this step.
3. Prompt Bob to create a SonarQube project and token:

> ⚠️ **IMPORTANT:** Before executing the command below, replace *${USER}* with your actual username (e.g., *user1*, *user2*, etc.) in the prompt. This ensures your SonarQube project is uniquely identified. Also replace *{REPLACE_SONARQUBE_PASSWORD}* with the credential provided by the instructor.

> *Execute the following to create a SonarQube token and project for ${USER}:*
>
> *TOKEN=$(curl -s -u txcdemo:{REPLACE_SONARQUBE_PASSWORD} -X POST "https://sonarqube-sonarqube.apps.itz-y6rg5f.infra01-lb.wdc07.techzone.ibm.com/api/user_tokens/generate?name=token-$(date +%s)" | jq -r .token); PROJECT_KEY=order-service-${USER}-$(date +%s); curl -u txcdemo:{REPLACE_SONARQUBE_PASSWORD} -X POST "https://sonarqube-sonarqube.apps.itz-y6rg5f.infra01-lb.wdc07.techzone.ibm.com/api/projects/create?project=$PROJECT_KEY&name=Order%20Service%20${USER}"; echo ""; echo "TOKEN=$TOKEN"; echo "PROJECT_KEY=$PROJECT_KEY"; echo "Project created in SonarQube!"*

4. Bob will run this command to:
   - Generates a unique SonarQube authentication token
   - Creates a unique project key with timestamp (e.g., *order-service-testuser1-1778261688*)
   - Creates the project in SonarQube via API
   - Displays the token and project key for the next step
5. Bob will return a token and project key. **Save both the TOKEN and PROJECT_KEY** - you'll need them for the next step.
6. Next, prompt to Bob generate an analysis report:

> *Scan order-service with the newly created SonarQube project and generate a comprehensive analysis report*

7. Bob will perform a comprehensive 5-step analysis process, which should include:
   - Bob will runs a scan using maven, which may fail the first time due to the injected vulnerabilities, but Bob will update its command and run something similar to: *cd order-service && mvn clean compile sonar:sonar -Dsonar.projectKey=order-service-testuser1-1778261688 -Dsonar.projectName="Order Service testuser1" -Dsonar.host.url=https://sonarqube-sonarqube.apps.itz-y6rg5f.infra01-lb.wdc07.techzone.ibm.com -Dsonar.token=squ_dc8b45225f4ebdcd40cc73a4a19d2b3b14f76755 -DskipTests*. Bob will automatically create the correct SonaryQube parameters based on the **PROJECT_KEY** and **TOKEN** from the prior step.

- Bob will fetch Issues from SonarQube API, using a command similar to: *curl -s -u txcdemo:{REPLACE_SONARQUBE_PASSWORD} "https://sonarqube.../api/issues/search?componentKeys=order-service&resolved=false&ps=500"*
- Bob will fetch security hotspots and vulnerability rating, using a command similar to: *curl -s -u txcdemo:{REPLACE_SONARQUBE_PASSWORD} "https://sonarqube.../api/hotspots/search?projectKey=order-service&ps=500"*
- Bob will fetch project metrics (i.e. quality gate metrics and rating) as well as bugs, vulnerabilities, code smells, coverage, technical debt, using a command similar to: *curl -s -u txcdemo:{REPLACE_SONARQUBE_PASSWORD} "https://sonarqube.../api/measures/component?component=order-service&metricKeys=..."*
- Finally, Bob will generate a comprehensive analysis report that includes: Executive summary with quality gate status, detailed breakdown of all issues, Security hotspots analysis with remediation code, Bugs and code smells with fixes, Comparison with manual security review, Remediation priority matrix, CI/CD integration recommendations, Quality gate configuration

Note: *Bob goes beyond just running the scan - it automatically fetches detailed data via API and creates a comprehensive markdown report with remediation guidance. This provides much more value than viewing results in the SonarQube UI!*

8. Open and review the generated *SONARQUBE_ANALYSIS_REPORT.md*. Some of the key findings you would see:
   - ✅ 100% Correlation - All 6 vulnerabilities identified by Bob were confirmed by SonarQube:
     - Hardcoded credentials ✓
     - Insecure logging ✓
     - Weak cryptography (MD5) ✓
     - Stack trace exposure ✓
     - Weak random generation ✓
     - Information exposure ✓
   - Additional SonarQube Findings:
     - Unused private fields
     - Unused method parameters
     - Generic wildcard usage
     - Math.abs edge case

Task 4: Add SonarQube & Grype Security Analysis Stage

> ***Note:*** *Before adding the security stage, ensure you have completed Part 3 (SonarQube token and project).*

In this section, we will add SonarQube to the Jenkins pipeline. As we have seen, we can use Bob to create the Jenkinsfile stage. You can leverage Bob's capabilities by crafting a **comprehensive, specification-style prompts** that describe exactly what you need. Instead of asking "Add security scanning," we will provide a complete technical specification with:

- ✅ Exact commands and tool versions
- ✅ Container contexts and file paths

- ✅ Data structure requirements (serializable types only)
- ✅ Error handling and retry logic
- ✅ Environment constraints (no external dependencies, rate limits)
- ✅ Success criteria and logging requirements

1. Switch to **Jenkins Pipeline Integration** mode before proceeding with this step.
2. Prompt Bob with the following:

> *Add a comprehensive Security Analysis stage to the Jenkinsfile that performs multi-layered security scanning. The stage should be placed after the existing stages and include:*
>
> *REQUIREMENTS:*
>
> *1. Environment Configuration:*
> *- Load SonarQube token from Jenkins credentials: credentials('sonarqube-token')*
> *- Set SONAR_HOST_URL to: https://sonarqube-sonarqube.apps.itz-y6rg5f.infra01-lb.wdc07.techzone.ibm.com*
> *- Set PROJECT_KEY dynamically using: "order-service-${env.USER}-${env.BUILD_NUMBER}"*
>
> *2. Phase 1 - SonarQube Security Scanning (MANDATORY):*
> *- Run Maven SonarQube scan from order-service directory*
> *- Use test-compile goal before sonar:sonar*
> *- **CRITICAL: Implement two-phase wait strategy:**
>
> * **Phase 1a - Wait for Task Completion (12 retries, 10 seconds between):**
> *- Check task status via: curl "${SONAR_HOST_URL}/api/ce/component?component=${PROJECT_KEY}"*
> *- Parse status using: grep -o '"status":"[^"]*"' | head -1 | cut -d'"' -f4*
> *- Continue only when status == 'SUCCESS'*
> *- Break if status == 'FAILED'*
> *- Use try-catch for error handling*
>
> * **Phase 1b - Fetch Metrics (6 retries, 5 seconds between):**
> *- Only after task completion, fetch metrics via: curl "${SONAR_HOST_URL}/api/measures/component?component=${PROJECT_KEY}&metricKeys=bugs,vulnerabilities,code_smells,security_hotspots,security_rating,reliability_rating,security_review_rating"*
> *- Parse JSON response using Groovy JsonSlurper (no jq dependency)*
> *- Store metrics as String primitives in securityResults.sonarqube map*
> *- Check if sonarData.component?.measures exists and has size > 0*
> *- Set sonarData = null after parsing to avoid serialization issues*
>
> *- Store results in serializable Map structure with String values*
> *- Display metrics in console output*
>
> *3. Phase 2 - Grype Vulnerability Scanning:*
> *- Install Grype on-demand in the build-tools container using: curl -sSfL https://raw.githubusercontent.com/anchore/grype/main/install.sh | sh -s -- -b /usr/local/bin*
> *- Run scan from order-service directory: grype dir:. --scope all-layers -o json > grype-report.json*
> *- Parse JSON to count Critical, High, and Medium severity vulnerabilities*
> *- Use grep and wc -l for counting (no jq dependency)*
> *- Store counts as Integer primitives in securityResults.trivy map*
> *- Wrap in catchError to prevent pipeline failure*
>
> *4. Phase 3 - CVE Analysis with Bob (conditional):*
> *- Only run if total vulnerabilities > 0*
> *- Use Bob in 'code' mode (not custom mode)*
> *- Provide detailed prompt including SonarQube and Grype results*
> *- Request: risk assessment, top 3 vulnerabilities, remediation steps, deployment recommendation*
> *- Save analysis to bob-cve-analysis.txt*

```
   - Extract deployment recommendation (BLOCK/WARN/PROCEED)

5. Phase 4 - Risk Level Calculation:
   - Parse SonarQube String values to Integer safely using Integer.parseInt()
   - Calculate risk score: vulnerabilities*10 + hotspots*5 + bugs*2 + critical_cves*20 + high_cves*10
   - Determine risk level: CRITICAL (≥50), HIGH (≥30), MEDIUM (≥10), LOW (<10)
   - Store as serializable primitives (String/Integer only)

6. Phase 5 - Deployment Decision:
   - CRITICAL → BLOCK deployment, set currentBuild.result = 'UNSTABLE'
   - HIGH → WARN with manual review recommendation
   - MEDIUM/LOW → PROCEED with approval

7. Phase 6 - Consolidated Report Generation:
   - Create markdown report: security-analysis-report.md
   - Include: executive summary, SonarQube results table, Grype results, CVE analysis, deployment recommendation
   - Archive artifacts: security-analysis-report.md, bob-cve-analysis.txt, order-service/grype-report.json

IMPORTANT CONSTRAINTS:
- Use only serializable data types (String, Integer, Boolean) in securityResults Map
- No jq or external JSON parsers - use Groovy JsonSlurper for SonarQube, grep/wc for Grype
- Install Grype in build-tools container (no separate security-tools container due to Docker Hub rate limits)
- Use 15-minute timeout for the entire stage
- Wrap all phases in catchError blocks to ensure pipeline completion
- Use container('build-tools') for Grype installation and scanning
- Use container('bob') for CVE analysis
- **CRITICAL: Must implement two-phase SonarQube wait (task status check THEN metrics fetch)**
- Store SonarQube metrics as Strings, parse to Integer only when calculating risk score
- Clear JSON objects after parsing (set to null) to avoid serialization issues

The stage should provide clear console output with emoji indicators and progress messages for each phase.
```

3. Bob generates a production-ready *Security Analysis* stage (for details on the structure see -). Some of the key benefits of this approach:
   - ✅ Mandatory SonarQube Integration - Ensures consistent security scanning across all builds
   - ✅ On-Demand Tool Installation - Installs Grype dynamically to avoid Docker Hub rate limits
   - ✅ Intelligent CVE Analysis - Uses Bob's AI for risk assessment and remediation guidance
   - ✅ Conditional Analysis - Only runs detailed CVE analysis when vulnerabilities detected
   - ✅ Comprehensive Risk Assessment - Considers ALL security reports together
   - ✅ No External Dependencies - Uses Groovy JsonSlurper and grep/wc (no jq required)
   - ✅ Clear Deployment Decisions - Automated risk-based gates with BLOCK/WARN/PROCEED logic

> ***Important:*** *You will add the SonarQube token to Jenkins credentials in a later step. The Jenkinsfile you create in this step will reference the credential ID* `sonarqube-token`*, which you'll configure before running the build.*

4. While this lab focuses on SonarQube as the mandatory security scanning tool, Bob's Software Security Reviewer mode (covered in Part 2) can be integrated into your pipeline for more comprehensive security analysis. However, this would require additional customization to fit your specific needs and workflow. The Software Security Reviewer mode is best used for:
   - Pre-deployment security audits
   - Compliance assessments (PCI DSS, OWASP)

- Threat modeling and attack scenario analysis
- Custom security requirements beyond standard code scanning

5. [OPTIONAL] If you wish to integrate the Software Security Reviewer mode into your pipeline, you can customize the security stage to include Bob-based analysis alongside SonarQube scanning.

Task 5: Push and Watch

1. Lets commit and push the updates. Prompt Bob or run the following commands in the terminal:

```
git add Jenkinsfile .bob/ labs/lab3/ order-service/src/main/java/com/example/orders/service/OrderService.java order-service/pom.xml
git commit -m "Lab 3 — Security Analysis stage with vulnerability scanning and injected vulnerabilities for testing"
git push
```

2. This will:
   - Adds the updated Jenkinsfile with the Security Analysis stage
   - Adds any custom Bob modes from .bob/ directory
   - Adds lab3 files (scripts, documentation)
   - **Adds OrderService.java with injected vulnerabilities** for security testing
   - **Adds pom.xml with vulnerable Log4j dependency** for dependency scanning
   - Commits all changes with descriptive message
   - Pushes to trigger Jenkins build with vulnerabilities present

> ***Note:*** *We're intentionally committing the files with vulnerabilities so the Jenkins pipeline can detect them during the security scan. We'll remove these vulnerabilities and push clean code later in the lab.*

3. Before running the build, you need to add your SonarQube token to Jenkins credentials so the pipeline can authenticate with SonarQube.
4. Navigate to Jenkins Credentials:
   - Go to Jenkins → **Manage Jenkins** → **Credentials**
   - Click **Add Credentials**
5. Create the credential:
   - **Kind:** Secret text
   - **Scope:** Global
   - **Secret:** Paste your SonarQube token from Step 3.1 (e.g., *squ_dc8b45225f4ebdcd40cc73a4a19d2b3b14f76755*)
   - **ID:** *sonarqube-token*
   - **Description:** SonarQube authentication token
   - Click **Create**
6. Run the Build:
   - Navigate to your pipeline in Jenkins
   - Click **Build Now**
   - Watch the console output as the pipeline executes
7. You should see the following:

- *Checkout* and *PR Review* stages turn green
- *Security Scan* stage runs with multiple security checks:
  - Secret scanning detects hardcoded credentials
  - SonarQube analysis identifies security hotspots
  - Dependency vulnerability scan runs
  - Code pattern checks flag insecure practices
  - Configuration security checks validate K8s/Docker configs
  - Risk assessment calculates overall security posture
- Security reports are generated and archived as build artifacts:
  - *Security_Analysis_Report.md* - Bob's comprehensive security analysis
  - *SonarQube_Analysis_Report.md* - SonarQube findings
  - *CVE_ANALYSIS_REPORT.md* - CVE analysis (if vulnerabilities detected)
  - *SECURITY_REPORT.txt* - Pipeline security summary
- Pipeline may turn **UNSTABLE (yellow)** or **FAILURE (red)** depending on severity of findings
- Build page lists all security reports under **Build Artifacts**

8. Open the archived artifacts from the build page to review:
   - Bob's detailed security analysis with remediation guidance
   - SonarQube's quality gate status and security hotspots
   - CVE analysis with risk assessment and deployment recommendations

Note: *In an ideal production environment, you would generate separate reports for each tool (SonarQube_Analysis_Report.md, CVE_ANALYSIS_REPORT.md, SECURITY_REPORT.txt). However, for demo purposes, this lab consolidates all findings into a single* security-analysis-report.md *that includes:*

- *Executive summary with risk level and deployment decision*
- *SonarQube analysis results (bugs, vulnerabilities, security hotspots, code smells)*
- *Grype vulnerability scan results (Critical, High, Medium severity counts)*
- *Risk assessment calculation and thresholds*
- *Deployment recommendation (BLOCK/WARN/PROCEED)*
- *Next steps based on risk level*

Note: *The security stage is designed to block deployment if CRITICAL issues are detected. Review the console output and security reports to understand what needs to be fixed before the next deployment.*

Task 6: Restore Original Code

After the Jenkinsfile changes have been pushed and the build has run, we need to remove the vulnerabilities from OrderService.java and pom.xml that were committed to the repository.

1. Switch to **Agent mode** and prompt Bob with:

> *Revert OrderService.java and pom.xml to their original state before vulnerability injection by checking out the version from before the vulnerabilities were added, then commit and push these changes*

2. Bob will:

  - Identifies the commit before vulnerabilities were injected
  - Restores the clean versions of the files:
    - *git checkout <commit-hash> -- order-service/src/main/java/com/example/orders/service/OrderService.java*
    - *git checkout <commit-hash> -- order-service/pom.xml*
  - Commits the restored files:
    - *git add order-service/src/main/java/com/example/orders/service/OrderService.java order-service/pom.xml*
    - *git commit -m "Restore OrderService.java and pom.xml to clean state - remove injected vulnerabilities"*
  - Pushes the changes:
    - *git push*

3. If you prefer to revert the entire vulnerability injection commit:

```
git log --oneline  # Find the commit hash of vulnerability injection
git revert <commit-hash>  # Creates a new commit that undoes the changes
git push
```

4. In Jenkins, click **Build Now** on your pipeline and watch the console.
5. This time, the pipeline should complete.

Information and Tips

- There is a cleanup script in the repo that restores the changes made to *OrderService.java* and removes the backup. You can run the script by asking bob: *Run script restore_vulnerabilities.sh*
- If you get stuck, you can restore changes that have not been committed yet using Git. For example, to revert a change made to the Jenkinsfile, use the command: *git checkout Jenkinsfile*
- Details for the *Security Analysis* stage you are adding can be found in the security-stage-details.md

Part 4 — Linting and Compliance with Bob

In this portion of the lab, you'll add a multi-tool linting and compliance workflow to the pipeline. All the while using IBM Bob to interpret results and provide recommendations.

- You'll use the Checkstyle for Java linter to find and fix Java code quality issues in the IDE.
- You'll use Hadolint to find issues in the *Dockerfile*.
- You'll use Checkov to find issues in the deployment manifests.
- You'll use KubeLinter to find issues in the Kubernetes workload.

> ***Note:*** *The* order-service/pom.xml *already includes Checkstyle configuration with Google's style checks. This lab leverages that existing configuration for both IDE and pipeline linting.*

This lab intentionally makes use of slightly flawed deployment files with predictable findings. That allows us to focus on using AI to:

- interpret the linter findings
- anticipate likely pipeline findings before Jenkins runs
- recommend fixes
- produce a consolidated lint report

What you'll build

By the end of this lab, you will use Bob to not just interact with linters — but it will explain what matters, which issues overlap across tools, what to fix first, and how to remediate them.

- **A Bob mode for lint reporting** (*pipeline-lint-analyzer*) — a read-only mode that reads the raw output from multiple linters and turns it into a Jenkins-friendly report.
- **A new set of pipeline stages** — one stage runs the linters (including Checkstyle), one stage asks Bob to summarize the results.
- **A final Bob-enriched lint report** — archived in Jenkins as a build artifact and summarized in a comment artifact.

Key Concepts you will learn:

- Linting layers: code quality, container security, infrastructure security, Kubernetes best practices
- De-duplication: recognizing when multiple tools flag the same underlying issue
- Risk-based prioritization: security > reliability > maintainability > style

Before you start

- ☐ You should have completed Lab 1 at a minimum (your Jenkinsfile already has the *askBob()* helper)
- ☐ You're on your working branch (for example *user1-labs*)
- **Reset your Jenkinsfile first.** Remove any stages you added in previous labs, keeping only the *Checkout* stage and the *askBob* helper at the bottom. Stacking every lab's stages stretches a single build past 15

minutes — clearing them keeps iteration fast and the logs readable. Use *solution/Jenkinsfile.start* as the reference shape.

Task 1 — Code Linting with Checkstyle

Before adding linters to our pipeline, lets start in the Bob IDE with some high level linting tasks at the code level.

1. If we were starting in a new project / application, we could actually have Bob review our application and help us identify linters already in place and provide recommendations for additional linters to consider including.
2. Start a new task in Bob and ensure you are in the **Ask** mode. Then prompt Bob with the following:

> *Analyze the files in my @/order-service application and recommend what linters I should use in my build and deploy pipeline*

3. Bob will review what linters are in place and what gaps might exist based on the files in the application. Review Bob's recommendations.
4. Lets narrow in on a specific linter. Start a new task in Bob and ensure you are in the **Ask** mode. Then prompt Bob with the following:

> *Read @order-service/src/main/java/com/example/orders/service/OrderService.java.*
> *Give me a concise view of what kinds of issues Checkstyle will likely flag here, which ones matter most, and what would you recommend fixing first?*

5. Bob itself understands different linting frameworks and can be used to get intuition about what issues Checkstyle will likely flag here, which ones matter most, and what it recommends fixing first. Review Bob's insights.
6. Next, lets actually run Checkstyle from the command line to generate a report with all findings. From the terminal window, execute the following:

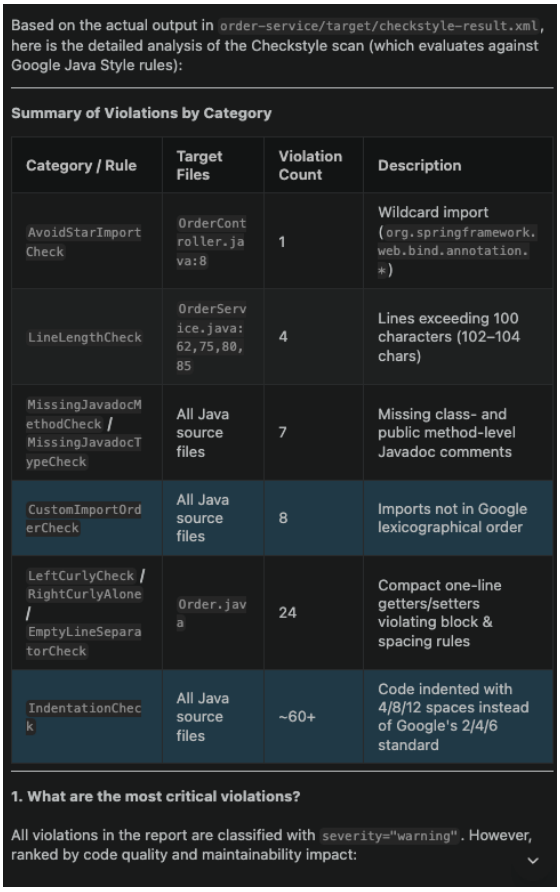
```
cd order-service
mvn checkstyle:check
```

7. This will generate a full report at *order-service/target/checkstyle-result.xml*. Lets ask Bob to analyze the output. In **Ask** mode, prompt Bob with the following:

> *I ran `mvn checkstyle:check` in the order-service directory. Read the output and explain:*
> *1. What are the most critical violations?*
> *2. Which ones could impact production reliability?*
> *3. What should I fix first?*
>
> *Only analyze the actual checkstyle output*

> ***Note:** Bob will request permission to read the target directory and will find the correct output report xml file.*

8. Review the findings from Bob. Notice how Bob:
   - Groups by severity
   - Explains the impact of each issue
   - Provides specific line numbers
   - Suggests concrete fixes
   - Prioritizes what matters most



Based on the actual output in `order-service/target/checkstyle-result.xml`, here is the detailed analysis of the Checkstyle scan (which evaluates against Google Java Style rules):

Summary of Violations by Category

Category / Rule	Target Files	Violation Count	Description
`AvoidStarImportCheck`	`OrderController.java:8`	1	Wildcard import (`org.springframework.web.bind.annotation.*`)
`LineLengthCheck`	`OrderService.java: 62,75,80, 85`	4	Lines exceeding 100 characters (102–104 chars)
`MissingJavadocMethodCheck` / `MissingJavadocTypeCheck`	All Java source files	7	Missing class- and public method-level Javadoc comments
`CustomImportOrderCheck`	All Java source files	8	Imports not in Google lexicographical order
`LeftCurlyCheck` / `RightCurlyAlone` / `EmptyLineSeparatorCheck`	`Order.java`	24	Compact one-line getters/setters violating block & spacing rules
`IndentationCheck`	All Java source files	~60+	Code indented with 4/8/12 spaces instead of Google's 2/4/6 standard

1. What are the most critical violations?

All violations in the report are classified with `severity="warning"`. However, ranked by code quality and maintainability impact:

9. You just ran *mvn checkstyle:check* and asked Bob to analyze the findings. This is a common pattern that we can encode as a custom slash command. Start a new task, switch to **Agent** mode and prompt Bob:

> *Create a custom slash command at .bob/commands/lint-check.md. The filename becomes the command name, so this defines `/lint-check`.*
>
> *When invoked, the command should:*
> *1. Run `mvn -f order-service/pom.xml checkstyle:check`*
> *2. Read `order-service/target/checkstyle-result.xml`*
> *3. Group findings by severity, name the fix-first items, and suggest concrete remediations*

10. Run your new command in a new task to confirm it works:

> */lint-check*

You should get the same analysis you got from the manual prompt — but in one step.

Bob was not only able to find and read the report, it was able to provide immediate feedback for which are the most critical issues, which impact reliability and a prioritization for fixing the issues. We can use these insights to get ahead of any linting issues before our applications even hit our pipelines.

> ***Note:*** *The prompts we are using can be adapted based on different perspectives, goals and end users.*

Task 2 — Deployment Artifact Linting

Beyond linting of the code artifacts, we can use Bob to inspect the deployment artifact files that are used to deploy this application and that the pipeline will scan. For the purposes of this lab, we have created a simplified set of artifacts for a Red Hat OpenShift deployment. The following files have been created with some "flaws", so that we can analyze them in this lab.

- *order-service/Dockerfile*
- *order-service/deploy-flawed/deployment.yaml*
- *order-service/deploy-flawed/service.yaml*
- *order-service/deploy-flawed/route.yaml*

1. Start a new task and switch to **Ask** mode. Try a prompt Bob with the following:

> *Read @order-service/Dockerfile and the manifests under @order-service/deploy-flawed/.*
> *What issues are likely to be flagged by Hadolint, Checkov, or KubeLinter?*
> *Group them by file and explain which ones are most important to fix first.*

2. Bob gives you immediate feedback and points out issues even before the linters have been run. Bob should identify issues, such as:
   - use of *latest* in container images
   - plaintext secrets or credentials in env vars
   - missing probes
   - missing resource requests and limits
   - weak or incomplete *securityContext*
   - overly permissive route or service settings
3. That prediction step is useful for two reasons:
   1. it helps you understand what the pipeline linters are likely to find
   2. it gives you a baseline to compare with the actual lint output later

While we could fix these issues now, we will leave these "flaws" in place to see how we can use Bob + various linters to identify the issues and suggest the appropriate fixes in a CI/CD pipeline.

Task 3 — Create the Bob Linter mode and Pipeline Stage

Now, we will create a reusable Bob mode that specializes in lint analysis, insights and remediation recommendations. While we could create a mode to use in our IDE, we are going to focus on creating a mode that can be used in a CI/CD pipeline. The pipeline needs a read-only mode that can synthesize raw lint output from multiple tools and turn it into a concise report.

> Note: While we could use the mode in the Bob IDE, we want to minimize any requirements for installing linters to the lab participant machines, so we are just going to test in the pipeline.

1. Start a new task and switch to **Mode Writer** mode. Then ask Bob to create a new mode with a name *pipeline-lint-analyzer* that it should append to our *@.bob/custom_modes.yaml* file. Use this as a starter prompt:

> *Write a custom mode with slug `pipeline-lint-analyzer`. Append it to @.bob/custom_modes.yaml — don't overwrite anything else.*
>
> *Job: read lint output from Checkstyle, Hadolint, Checkov, and KubeLinter, plus the relevant application source files and application deployment files.*

> *Produce a short Jenkins-friendly report that:*
> *- groups findings by severity*
> *- identifies overlap across tools*
> *- names the most important fix-first items*
> *- suggests concrete remediations*
> *- stays concise and practical*
>
> *Output: plain text or Markdown suitable for a Jenkins artifact and easy to summarize in a PR comment.*
> *Use the following sections:*
> *- Executive Summary*
> *- Highest Priority Findings*
> *- Findings by Tool*
> *- Cross-Tool Themes*
> *- Recommended Fix Order*
>
> *If findings are duplicated across tools, call that out instead of repeating the same issue many times.*
>
> *Tool groups: read only.*

> ***Tip:** This prompt is a good starting point that has been tested with the pipeline, but feel free to extend the prompt above with your own preferences.*

2. In Part 1 of this lab, you ran Checkstyle manually and saw how Bob can interpret its findings. Now we'll automate this in the pipeline along with other linters. So next, lets add the linting and analysis stages to the pipeline.
3. Start a new task and switch to the provided **Jenkins Pipeline Integration** mode.
4. Ask Bob to update *Jenkinsfile* with a lint workflow after the earlier stages. Use a prompt like:

> *Update @Jenkinsfile to add linting and analysis stages after the existing stages.*
>
> *Requirements:*
> *- Run Checkstyle on the Java code: `mvn -f order-service/pom.xml checkstyle:check`*
> *- Copy the XML report from `order-service/target/checkstyle-result.xml` to `lint-results/checkstyle.xml`*
> *- Run Hadolint on @order-service/Dockerfile and save output to `lint-results/hadolint.txt`*
> *- Run Checkov on @order-service/deploy-flawed/ and save output to `lint-results/checkov.txt`*
> *- Run KubeLinter on @order-service/deploy-flawed/ and save output to `lint-results/kubelinter.txt`*
> *- Continue the pipeline even if the linters find issues (use catchError)*
> *- Archive the lint-results files in the Run Linters stage post block*
>
> *For the Bob Lint Analysis stage:*
> *- Add a `when` condition to only run if at least one lint result file exists*
> *- In the prompt, let Bob read the files directly - don't embed file contents in the prompt. Tell Bob to read:*
> *- lint-results/checkstyle.xml*
> *- lint-results/hadolint.txt*
> *- lint-results/checkov.txt*
> *- lint-results/kubelinter.txt*
> *- order-service/src/*
> *- order-service/Dockerfile*
> *- order-service/deploy-flawed/*
> *- Make a call using askBob using the `pipeline-lint-analyzer` mode to group findings by severity, de-duplicate overlapping findings, identify highest-priority remediations, and recommend concrete fixes*
> *- Save Bob's analysis to `bob-lint-report.md`*
> *- Make a second askBob call that reads `bob-lint-report.md` and produces a concise PR comment*
> *- Save the PR comment to `bob-lint-pr-comment.md`*
> *- Archive both markdown files in the post block*
> *- Print a short summary to the console*

> ***Tip:*** *We are asking Bob to create a "PR comment", which is a concise summary of the lint report. Though not currently implemented, this comment could be attached to a PR and displayed to the reviewer.*

5. A couple of things to note about how we are asking Bob to build this into our pipeline:
   - We are asking Bob to split the work into the following stages:
     - *Run Linters* (runs Checkstyle, Hadolint, Checkov, KubeLinter)
     - *Bob Lint Analysis* (analyzes all findings)
     - *Post Lint PR Comment* (posts summary to GitHub)
   - The linters should run in their appropriate containers:
     - Checkstyle runs in the *build-tools* (Maven) container
     - Hadolint, Checkov, and KubeLinter run in the *lint-tools* container
   - The pipeline should produce files like:
     - *lint-results/checkstyle.xml* (or checkstyle.txt)
     - *lint-results/hadolint.txt*
     - *lint-results/checkov.txt*
     - *lint-results/kubelinter.txt*
     - *bob-lint-report.md*
     - *bob-lint-pr-comment.md*
   - Jenkins takes on the following responsibiltiies:
     - running the tools
     - saving files
     - archiving artifacts
   - Bob takes on the following responsibilities:
     - grouping findings
     - prioritizing fixes
     - suggesting remediation
     - generating the report text
6. Feel free to open and review the Jenkinsfile (optionally, you could even ask Bob to summarize the changes in the Jenkinsfile to you).

Task 4 — Push and watch

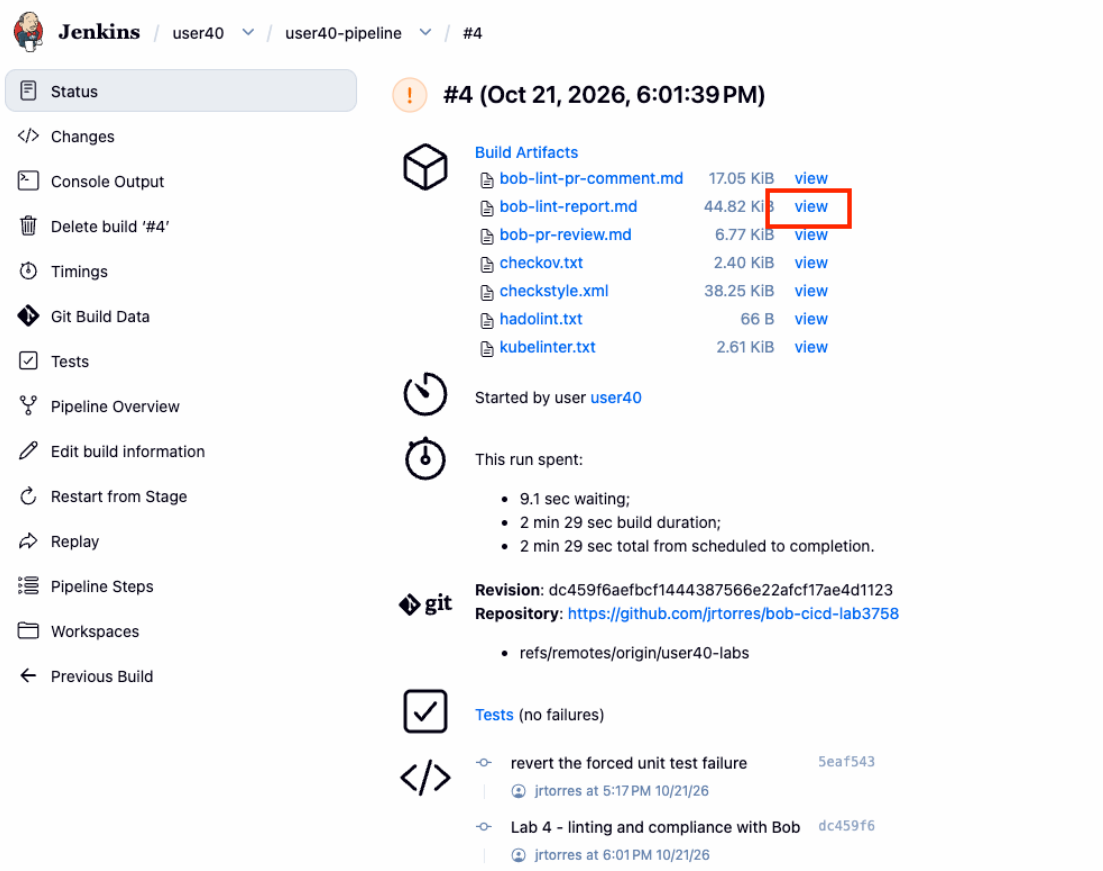
1. Now that your Jenkinsfile and mode definitions are ready, let push them to your repo.

```
git add Jenkinsfile .bob/
git commit -m "Lab 4 — linting and compliance with Bob"
git push
```

2. Back in the Jenkins instance, click **Build Now** and watch the updated pipeline run.
   - Expected behavior:
     - earlier stages still run
     - Checkstyle runs in the Maven container
     - the other lint tools run inside the *lint-tools* container
     - raw outputs are written under *lint-results/*

- Bob prints a short summary in the console
- bob-lint-report.md is archived as a build artifact
- the build may end **UNSTABLE** if findings exist, but should still complete

3. Open the pipeline run artifacts, you should see a consolidated report that is far more useful than the raw scanner output alone.



When you're ready, continue to JENKINS_MCP.md.

Part 5 — Jenkins Pipeline Insights with MCP

In this portion of the lab, you'll configure the Jenkins MCP server to interact with your Jenkins instance directly from Bob, enabling pipeline monitoring, build analysis, and automated insights without leaving your IDE. This is a simpler lab and does not involve any modifications to your Jenkins instance or the pipeline you have built throughout the workshop. The lab will give you a sense for life with and without the MCP server

Without the Jenkins MCP server:

- Switch to Jenkins UI to check build status
- Manually search through console logs
- Copy/paste log snippets into Bob for analysis
- Context-switch between tools

With the Jenkins MCP server:

- Query pipeline status from Bob: "What's the status of my last 5 builds?"
- Analyze failures in context: "Why did build #42 fail?"
- Generate insights: "Show me all failed builds from today"
- Automate monitoring: "Alert me if the success rate drops below 80%"
- Correlate with code: "Which commit caused build #42 to fail?"

> ***Note:***
>
> - It is assumed you have completed some or all the previous labs so that there will be pipelines and pipeline runs in the instance.
> - The Jenkins MCP server is configured using *mcp-jenkins* via *uvx*.

What you'll build

In this lab, you won't build anything, but you will use the Jenkins MCP server to interact with your Jenkins instance directly from Bob. You will use the MCP server to explore:

- **Pipeline discovery tools** — query all pipelines on your Jenkins instance, explore their configurations, and understand their structure.
- **Build analysis capabilities** — inspect specific builds, retrieve logs, analyze test results, and identify failure causes.
- **Automated insights** — generate reports on pipeline health, identify patterns in failures, spot flaky tests, and correlate issues with code changes.

By the end, you'll be able to monitor and analyze your Jenkins pipelines entirely through Bob, automating common queries and generating actionable insights.

Before you start

- ☐ Lab 1 complete (you understand Jenkins pipeline structure and have a working pipeline)
- ☐ Jenkins credentials provided by your instructor (URL, username, API token)

Task 1 — Jenkins MCP Server

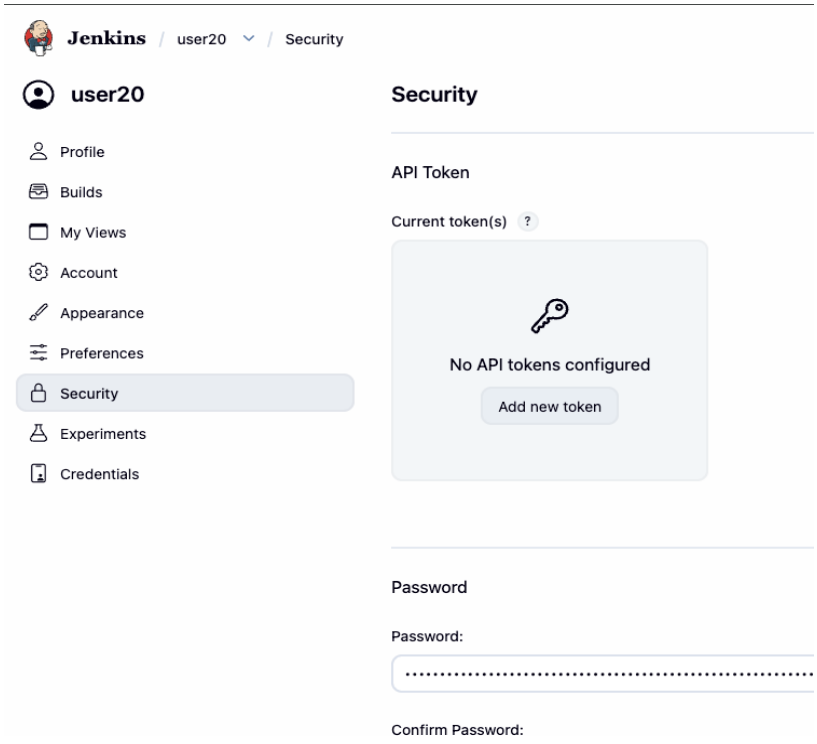
The Jenkins MCP server is a Model Context Protocol server that connects Bob to your Jenkins instance through the Jenkins REST API. It provides tools that let Bob query pipelines, inspect builds, retrieve logs, and analyze results — all through natural language.

The *mcp-jenkins* server provides these following tools (just a subset of the available tools):

- ***list_jobs*** — List all pipelines/jobs on the Jenkins instance
- ***get_job_info*** — Get detailed information about a specific job (configuration, parameters, recent builds)
- ***list_builds*** — List recent builds for a specific job
- ***get_build_info*** — Get detailed information about a specific build (status, duration, stages, artifacts)
- ***get_build_log*** — Retrieve the console output for a build
- ***get_build_test_results*** — Get test results from a build (passed, failed, skipped)

To configure the server, first lets setup an API key for the Jenkins instance.

1. Open your Jenkins URL in a browser and log in.
2. Click on your profile picture in the top right corner and select **Security** from the left panel.



3. Click on the **Add new token** button, then in the popup modal give your token a name, then click **Generate** button.
4. Copy the token value, you will need it later, and click the **Done** button.

Next, lets setup the MCP server. The *.bob/mcp.json* file already exists at the repo root.

1. Open the *.bob/mcp.json* file. If it already has other MCP servers configured, that's fine — we'll add the Jenkins server alongside them.
2. Add the Jenkins server block to the *mcpServers* object:

```
{
"mcpServers": {
  "jenkins": {
  "command": "bash",
  "args": [
    "-c",
    "set -e; set -a; source ${workspaceFolder}/.env; set +a; exec uvx mcp-jenkins --jenkins-url \"$JENKINS_URL\" --jenkins-username \"$JENKINS_USERNAME\" --jenkins-password \"$JENKINS_API_TOKEN\""

  ],
  "disabled": false,
  "alwaysAllow": []
  }
}
}
```

3. Save the *mcp.json* file.

Next, we have to add the credentials to our environment file.

1. Open the *.env* file at the repo root (create it if it doesn't exist; if it already has other credentials, add the Jenkins ones below them) and add these lines (update it with your username and the token you created in the previous steps):

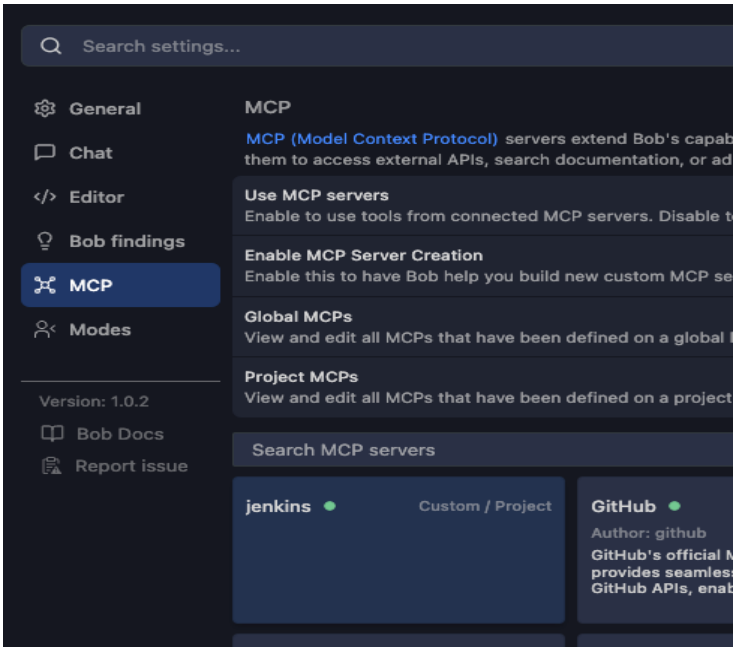
```
JENKINS_URL=https://jenkins.example.com
JENKINS_USERNAME=your-username
JENKINS_API_TOKEN=your-api-token
```

> ***Note:***
> - *You dont need quotes around the values, nor spaces around the =*
> - *The URL should include https:// but no trailing slash*
> - *.env is gitignored, so your token won't be committed*

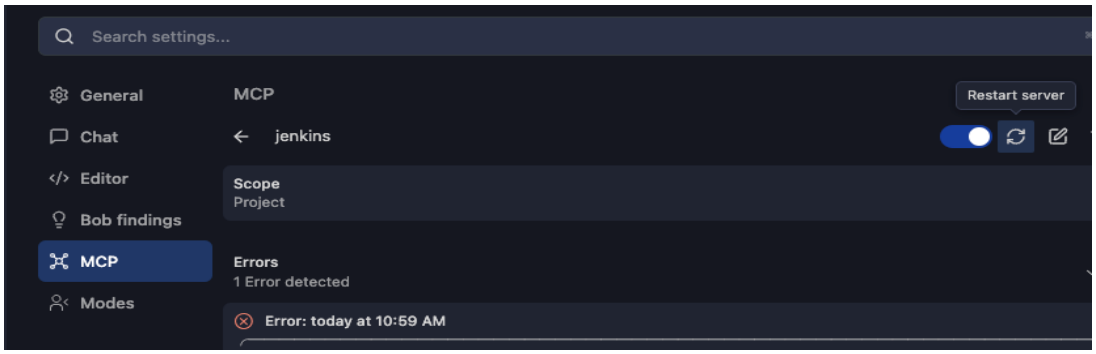
2. Save the *.env* file. and restart Bob (Bob reads *.bob/mcp.json* and *.env* once at startup).

Let's verify the MCP server started.

1. Open Settings (gear icon) near the top right of the Bob IDE, then click on the *MCP* option on the left panel



2. Look for *jenkins* in the MCP server list
3. Status should show **Connected** (green).
4. Just to be sure, restart the MCP server by clicking on the *jenkins* MCP server and then clicking the *Restart server* icon



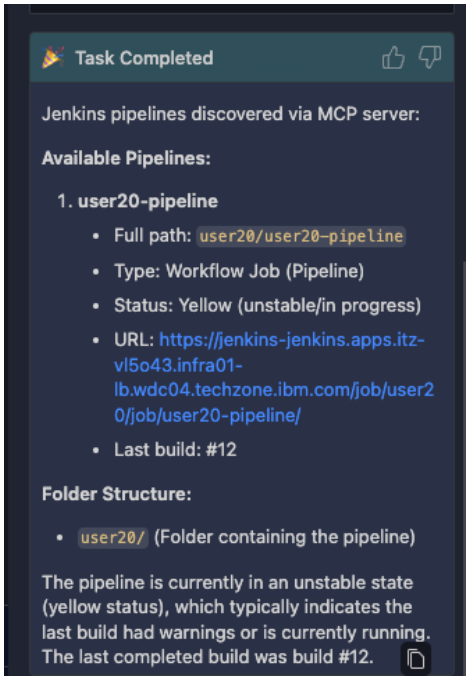
5. If you see **Failed**, check the following (do not proceed to the next section until you have successfully connected to the MCP server):
   - Double-check *.env* is at the repo root
   - Verify all three Jenkins values are filled in
   - Ensure no quotes or extra spaces
   - Check the Jenkins URL is accessible from your machine
   - Review the MCP server logs in Settings → MCP → jenkins → View Logs

Finally, lets run a quick test to make sure the MCP server is working.

1. Switch to **Agent mode** (MCP tools aren't available in Plan or Ask).

> *use the jenkins mcp server to check what pipelines are available*

2. If Bob says the jenkins server isn't available, go back to the previous step and verify the connection status. If Bob display a response with your pipeline, you are connected to the Jenkins instance



Task 2 — Pipeline Information

Now that the server is connected, let's explore what's on your Jenkins instance.

1. Switch to the **Agent mode**, ask Bob:

> *Use the jenkins MCP server to list all jobs/pipelines on the system.*

2. Bob will call the *get_all_item* tool and show you all pipelines. You should see your pipelines (e.g., *user1/user1-pipeline*).
3. Now focus on your specific pipeline. Replace *user1/user1-pipeline* with your actual pipeline name and run:

> *Use the jenkins MCP server to get detailed information about the user1/user1-pipeline pipeline.*

> ***Note:** we are using the folder/pipelinename syntax to specify the pipeline. It should match what was returned in step 1 of Part 2.*

4. Bob will call several tools to provide a comprehensive summary of your pipeline including details like: Pipeline description, Last build number and status, Recent build history, Build artifacts (if any), and more.
5. Spend some time exploring the response from Bob.
6. Ask Bob to show you the recent builds for this pipeline (remember to replace the pipeline name with your own pipeline name):

> *Use the jenkins MCP server to list the last 10 builds for the job "user20-pipeline".*

7. Bob will use the *get_build* tools to show information about the pipeline builds, including status, duration, timestamps, etc. Along with a summary of success rate and its own observations.
8. Finally, lets ask Bob to give us some analysis on the build information we have so far. Ask Bob to explain what it sees:

> *Based on the job information and recent builds, explain the structure and health of my pipeline.*

9. Bob will analyze the data and give you some trends around your pipeline builds, potential issues and recommendations for improvement.

Task 3 — Analysis and Insights

Now let's explore a couple of more complex / comprehensive scenarios we can explore with Bob.

1. Pick a recent build number from Part 2 that either had a *FAILURE* OR *UNSTABLE* (let's say build #12). Ask Bob to retrieve and analyze the logs (remember to replace the pipeline name with your own pipeline name)::

> *Use the jenkins MCP server to get the console log for build #12 of "user1-pipeline". Analyze it and tell me what caused the failure.*

2. To understand what changed between a successful and failed build, ask Bob to compare two build numbers (be sure to use a successful and a failed build that you saw from Part 2 and remember to replace the pipeline name with your own pipeline name).

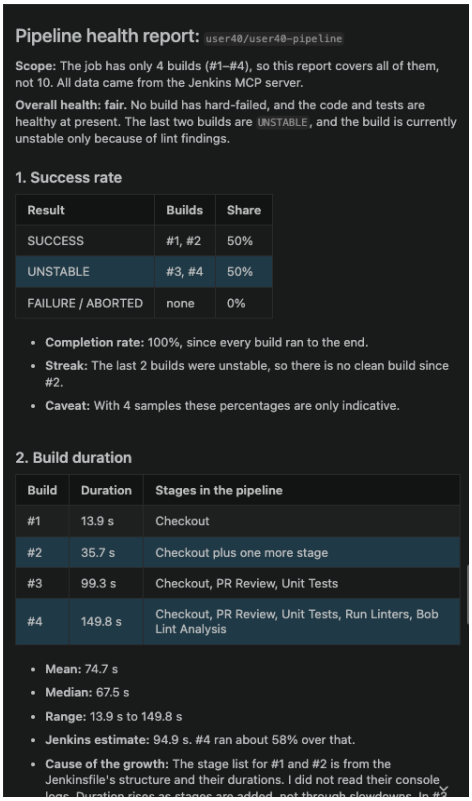
> *Compare build #14 (successful) with build #12 (failed) of "user1-pipeline". What changed?*

3. Bob will retrieve information for both builds and tell you what changed between the two builds. Focusing on: GIT comitts, files that changed, test differences, etc.
4. We can also ask Bob to do more than compare two builds, but provide analysis across various builds. Go ahead and ask Bob to look for patterns (remember to replace the pipeline name with your own pipeline name):

> *Use the jenkins MCP server to analyze the last 20 builds of "user1-pipeline". What patterns do you see in failures? What's the success rate? What's the average build duration?*

5. Again, Bob will retrieve data and provide its analysis. Providing things like success rate, failure patterns, average durations, etc.
6. Finally, lets ask Bob to create a comprehensive report for us using the following prompt (remember to replace the pipeline name with your own pipeline name):

> *Use the jenkins MCP server to create a health report for "user1-pipeline" covering the last 10 builds. Include:*
> *- Success rate*
> *- Average build duration*
> *- Most common failure causes*
> *- Test stability*
> *- Trends over time*
> *- Recommendations for improvement*



Pipeline health report: `user40/user40-pipeline`

Scope: The job has only 4 builds (#1–#4), so this report covers all of them, not 10. All data came from the Jenkins MCP server.

Overall health: fair. No build has hard-failed, and the code and tests are healthy at present. The last two builds are `UNSTABLE`, and the build is currently unstable only because of lint findings.

1. Success rate

Result	Builds	Share
SUCCESS	#1, #2	50%
UNSTABLE	#3, #4	50%
FAILURE / ABORTED	none	0%

- **Completion rate:** 100%, since every build ran to the end.
- **Streak:** The last 2 builds were unstable, so there is no clean build since #2.
- **Caveat:** With 4 samples these percentages are only indicative.

2. Build duration

Build	Duration	Stages in the pipeline
#1	13.9 s	Checkout
#2	35.7 s	Checkout plus one more stage
#3	99.3 s	Checkout, PR Review, Unit Tests
#4	149.8 s	Checkout, PR Review, Unit Tests, Run Linters, Bob Lint Analysis

- **Mean:** 74.7 s
- **Median:** 67.5 s
- **Range:** 13.9 s to 149.8 s
- **Jenkins estimate:** 94.9 s. #4 ran about 58% over that.
- **Cause of the growth:** The stage list for #1 and #2 is from the Jenkinsfile's structure and their durations. I did not read their console

Wrap up

You've completed the lab! You now have:

- ✅ Automated PR reviews (Part 1)
- ✅ Intelligent test failure analysis (Part 2)
- ✅ Security scanning with Bob (Part 3)
- ✅ Multi-tool linting and compliance (Part 4)
- ✅ Jenkins pipeline insights and automation (Part 5)

Next steps:

- Apply these patterns to your own projects
- Create custom modes for your team's workflows
- Explore other MCP servers (Slack, PagerDuty, Datadog, etc.)
- Share your Bob configurations with your team
- Build automation that combines multiple MCP servers

Congratulations on completing this lab! 🎉

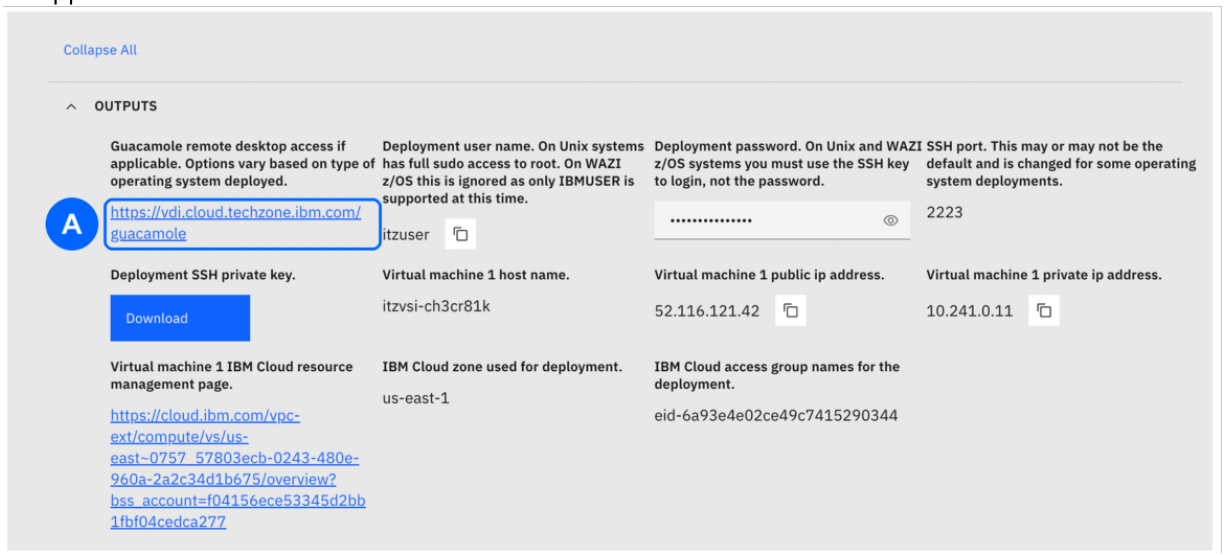
Accessing Lab Environment (Bob IDE)

A lab environment (virtual server instance) will be made available to you with IBM Bob already installed, along with the various tools required throughout the lab (i.e. Java, Python, Git, UV/UVX, and Maven). To access the environment, follow these instructions:

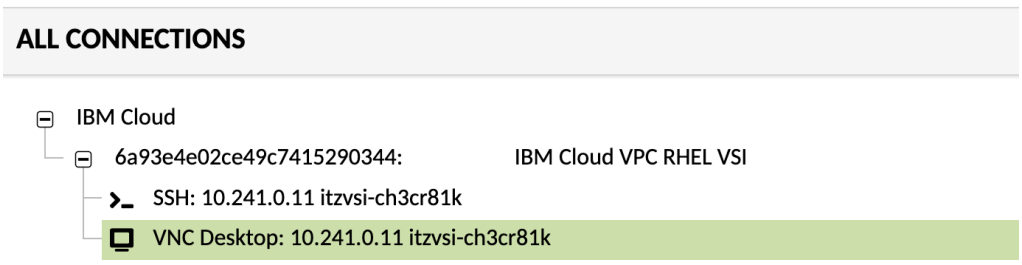
Connect to your lab image

The lab image is a Red Hat Enterprise Linux virtual machine with a desktop, which you reach through your own browser. It has Firefox, a terminal and the IBM Bob IDE and all the required dependencies of this lab already installed. Nothing runs on your laptop.

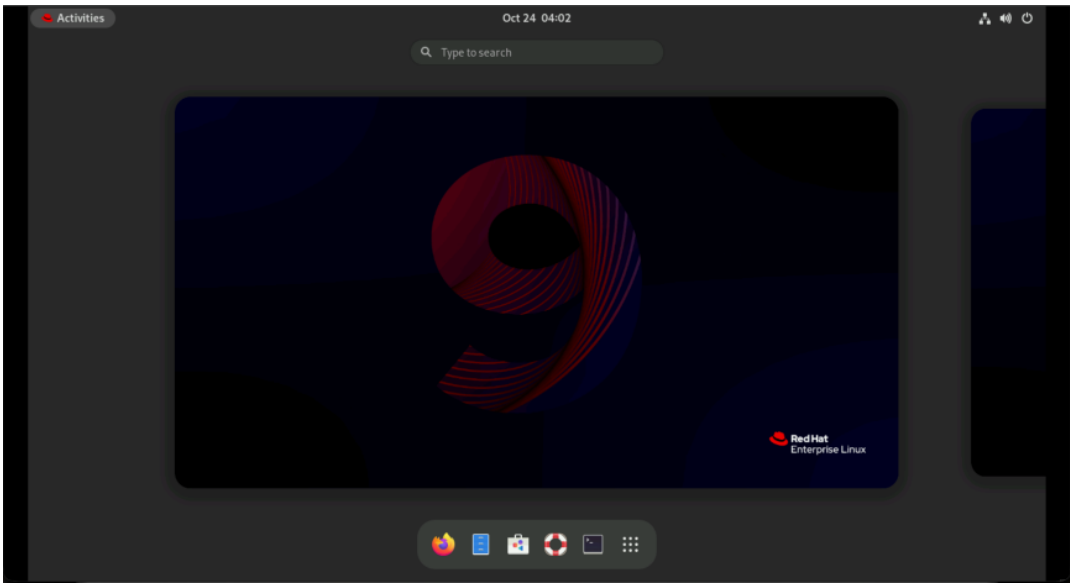
1. Open your TechZone lab details page, the one you land on after registering, and expand its Outputs.
2. Open the remote desktop access link (A). You can search for “Guacamole remote desktop access if applicable”.



3. Under All connections, choose the **VNC Desktop** entry, not the SSH one. SSH gives you a shell without a browser, and most of Part 1 happens in a browser. If you have multiple environments, locate your environment by Id from the previous screen.



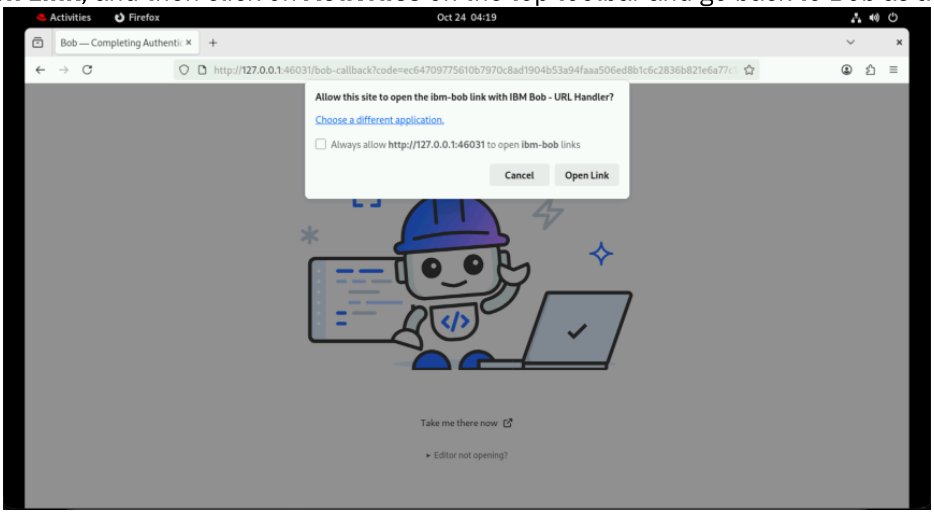
4. You should get a desktop with Firefox, a terminal and IBM Bob. Leave the browser open as you will use it for this lab.



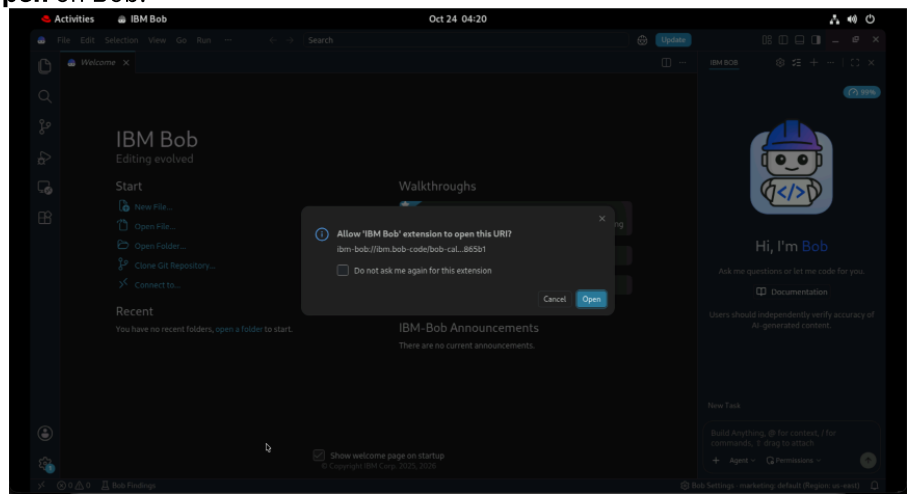
Launch Bob

In this step, you launch and login to Bob.

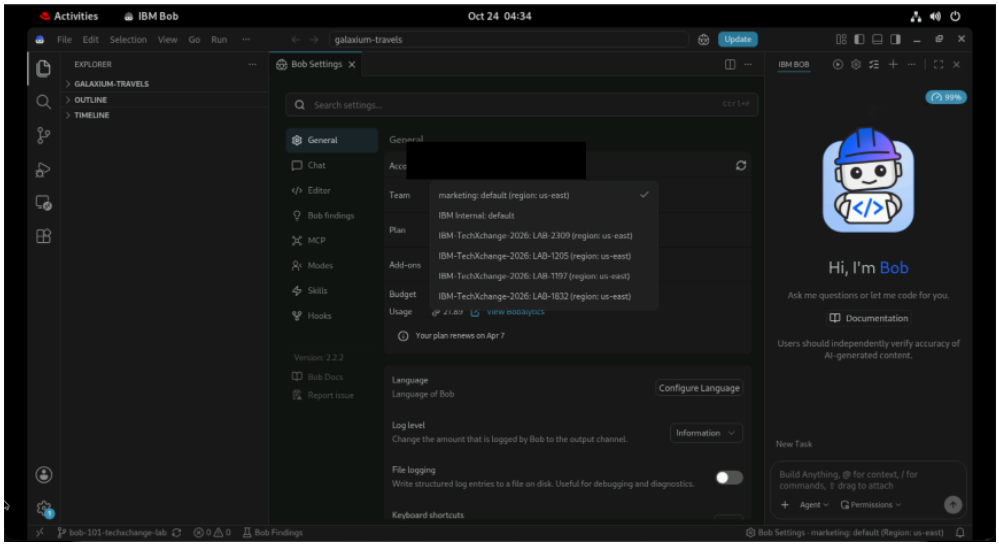
1. Click on **Activities** at the top of the screen to open the application launcher. Find and click IBM Bob to launch it.
2. Dismiss the Keyring Prompt. Bob will open and may display a keyring password dialog. Click **Continue twice** without entering a password as you do not need to set a keyring password in TechZone demonstration environments.
3. Click on **Log in to Bob**, then click on **Allow**, then click on **Open**. It will open a browser for you to login to Bob.
4. Firefox browser will be opened on the login page. Login to Bob using your IBMid on Firefox.
5. Click **Open Link**, and then click on **Activities** on the top toolbar and go back to Bob as authenticated**.**



6. Click on **Open** on Bob.



7. Click on **Bob Settings** at the very bottom toolbar and choose the team corresponding to your lab number.



Troubleshooting

If you are attending a proctored session, raise your hand or approach the instructor station. Lab instructors are available throughout the session to help with any issues. Below are some of the issues that may arise grouped by the portion of the lab they might appear in. Problems that can occur in any lab that adds a pipeline stage are listed first

All Pipeline Labs

- **Pipeline fails with something like *askBob: method not found*.** The helper function is inside the *pipeline { }* block. Move it to the top level of the Jenkinsfile, outside the *pipeline { }* braces.
- **The Bob stage runs but says the mode wasn't found.** Check that (a) *.bob/custom_modes.yaml* contains the mode's slug (for example *pipeline-git-diff-overview* or *pipeline-test-failure-analyzer*), (b) you're passing that exact string to *askBob*, and (c) you committed and pushed *.bob/*. Bob reads *custom_modes.yaml* fresh from the workspace on every run.
- **Bob container starts but CLI fails to run** — Confirm the Bob image is pushed to the jenkins namespace. Ask your instructor to verify the image URL in your Jenkinsfile pod spec.

Part 0 - Setup

- **Build fails with *ImagePullBackOff***: the Jenkinsfile's Bob image URL doesn't match your cluster. Ask your instructor to confirm the Bob image is pushed to the *jenkins* namespace.
- **"Delete Folder" button is missing from my folder**: that's intentional. Matrix auth doesn't grant delete rights on the folder itself so you can't accidentally wipe your workshop work.
- ***git push* fails with auth or a 403**: the PAT may have expired, been scoped to the wrong repository, or be missing **Contents: Read and write**. Create a new one (1b) and enter it when Git prompts. If the old one is cached, clear it with *git credential-cache exit*.
- **"Repository not found"** when cloning or building: double-check the repository URL, and that your fork exists and is public.
- **Git asks for username/password repeatedly**: you haven't enabled credential caching, or the 3-hour cache expired. See 1g.
- **Can't see files in Bob IDE**: make sure you opened the repository root directory (*bob-cicd-lab3758*), not a subdirectory.

Part 1 - PR / Git Diff Review with Bob

- **Part 2 stage works but the output is messy / inconsistent across runs.** Expected — that's the whole reason for Parts 4–6. Without a mode, you're relying on the prompt to enforce format every time, and the prompt in Part 2 is intentionally minimal.
- **The diff is empty.** Two common causes: (a) your branch hasn't diverged from *main* yet — there are no commits since the branch point — or (b) *origin/main* isn't available locally on the Jenkins agent (rare, but

possible if the Git plugin's clone config was customized). Confirm by adding *git branch -r* to your shell step temporarily: it should list *origin/main* after *checkout scm*. If it doesn't, the agent isn't fetching *main*; you may need an explicit *git fetch origin main* before the diff.

- **Pipeline fails with *fatal: detected dubious ownership in repository*.** Git refuses to operate when the directory's owner UID doesn't match the user running git. The *build-tools* (maven) container runs as a different user than the one that owns the checked-out files. Fix: add *git config --global --add safe.directory "$WORKSPACE"* as the **first** command in your stage's shell step, before any other git invocation. Use *$WORKSPACE* rather than the *'*'* wildcard so you're only trusting the directory you actually need.
- **Bob says *git-diff.txt* "is not accessible from the current workspace directory".** This means your stage wrote the diff to an absolute path outside Bob's reachable directory tree — almost always */workspace/git-diff.txt* instead of *git-diff.txt*. Bob's CWD is the Jenkins job workspace (*/workspace/workspace/<folder>/<job>/*), not the root of the shared volume; its tooling won't reach up out of its own workspace. Fix: write the diff to a **relative path** (*git-diff.txt*, no leading slash) and tell Bob to read the same relative path.
- **Bob output looks like a novel, not a triage summary.** The mode's rules file probably isn't constraining output length. Re-open Mode Writer and refine the rules to enforce brevity — cap per-change output to a few lines, require the three-section structure, tell Bob to skip uninteresting changes.

Part 2 - Unit Testing with Bob

- ***mvn test* fails the stage hard (red, not yellow).** The mode forgot to wrap the test step in *catchError*. Without it, Maven's non-zero exit code kills the pipeline before Bob ever runs. The *Jenkins Pipeline Integration* mode's rules cover this — re-prompt the mode to add *catchError(buildResult: 'UNSTABLE', stageResult: 'UNSTABLE')* around the *mvn test* call.
- ***junit* step says "no test reports".** The path is wrong. It should be *order-service/target/surefire-reports/*.xml* — verify the glob matches what *mvn test* actually wrote.
- **Bob's analysis runs even when tests pass.** Bob is unconditionally invoked. Move the *askBob* call inside an *if* that checks for failures (e.g., *if (currentBuild.result == 'UNSTABLE')*) so it only runs when there's something to diagnose.
- **Bob's output is too generic ("the test failed because the assertion failed").** The rules aren't constraining the analysis enough. Re-open Mode Writer and tighten the rules: require Bob to name the failing line, the expected vs. actual values, and a specific code-level fix.

Part 3 - Security Vulnerability Analysis

- **Tests fail after vulnerability injection** - This is expected behavior since the injected vulnerabilities break the validation logic. Use *-DskipTests* to continue: *mvn clean compile -DskipTests*
- **Bob can't find files** - Make sure you are in the correct directory. The command *pwd* should return this repo
- **Pipeline security stage fails** - Verify that SonarQube is accessible, your SonarQube Token is set (*echo $SONAR_TOKEN*), and that Maven can compile (*cd order-service && mvn compile*)

- **Bob completes the analysis but doesn't create a report file** - Use a follow-up prompt to ask Bob to create the missing reports (*Create the SECURITY_ANALYSIS_REPORT.md file with all the findings you just analyzed*)
- **SonarQube 401 Unauthorized** — Your SONAR_TOKEN credential is not configured in Jenkins. Add it under Manage Jenkins > Credentials with ID sonarqube-token.

Part 4 - Linting and Compliance with Bob

- **Checkstyle fails with *command not found*.** Checkstyle runs via Maven, not as a standalone command. Use *mvn checkstyle:check* in the Maven container, not the lint-tools container.
- **Checkov output is huge and noisy.** Narrow the scan target to *order-service/deploy-flawed/* rather than the whole repo.
- **KubeLinter reports nothing useful.** Make sure you are linting the deployment manifests, especially *order-service/deploy-flawed/deployment.yaml*.
- **Hadolint, Checkov, or KubeLinter fail with *command not found***
  - Cause: The lint-tools image wasn't built correctly or isn't being used
  - Fix: Verify the image from *setup/lint-tools/Dockerfile* was pushed
  - Check: *oc get imagestream lint-tools -n jenkins*
  - Verify: Jenkinsfile references the correct image URL
- **Pipeline succeeds but no lint report is generated**
  - Cause: Bob analysis stage didn't run
  - Check: Verify at least one lint-results/*.txt file exists
  - Check: Look at the *when* condition in "Bob Lint Analysis" stage
  - Debug: Add *sh 'ls -la lint-results'* to see what files were created

Part 5 - Jenkins Pipeline Insights with MCP

- 🔌 **MCP Server Won't Connect:**
  - Symptom: Settings → MCP shows jenkins server as "Failed" or "Disconnected"
  - Solutions:
    √ Verify *uv* is installed: *uv --version*
    √ Check *.env* file exists at repo root and has all three Jenkins values
    √ Ensure no quotes around values in *.env*
    √ Verify Jenkins URL is accessible: open it in a browser
    √ Check for typos in *.bob/mcp.json* (valid JSON?)
    √ Restart Bob completely (quit and reopen)
    √ Check MCP server logs: Settings → MCP → jenkins → View Logs
- 🔐 **Authentication Failures**
  - Symptom: MCP server connects but tools fail with "401 Unauthorized" or "403 Forbidden"
  - Solutions:
    √ Verify API token is correct (regenerate in Jenkins if needed)
    √ Check username matches your Jenkins login exactly

  - ✓ Ensure API token has appropriate permissions (read access to jobs)
  - ✓ Test credentials by logging into Jenkins UI
  - ✓ Verify Jenkins URL doesn't have a trailing slash in *.env*
- 🔍 **"Job not found" Errors**
  - **Symptom:** Bob says the job/pipeline doesn't exist
  - **Solutions:**
    ✓ Verify job name matches exactly (case-sensitive)
    ✓ Use *list_jobs* first to see available job names
    ✓ Check you have permissions to access the job in Jenkins UI
    ✓ Ensure the job exists (wasn't deleted or renamed)
    ✓ Try the full job path if it's in a folder (e.g., "folder/job-name")
- ⚙️ ***uvx* Command Not Found**
  - **Symptom:** MCP server fails to start with "uvx: command not found"
  - **Solutions:**
    ✓ Install *uv*: Follow installation instructions
    ✓ Verify installation: *uv --version*
    ✓ Restart terminal after installation
    ✓ Check *uv* is in your PATH: *which uv*
    ✓ Try running *uvx --version* directly

Additional Resources

- IBM Bob
  - Bob documentation
  - Bob IDE documentation
  - Custom modes guide
- Git and GitHub
  - Git documentation
  - GitHub forking guide
- Security
  - SonarQube security rules for Java
  - OWASP Top 10
  - CWE Top 25
  - PCI DSS requirements
  - Trivy: container vulnerability scanner
  - OWASP Dependency-Check: dependency vulnerabilities
  - Checkmarx: SAST scanning
- Model Context Protocol
  - Model Context Protocol
  - uv installation

Feedback

We value your feedback! Please take a few minutes to complete the TechXchange 2026 lab survey using the QR code or link provided by your lab instructor. Your feedback helps us improve future sessions.